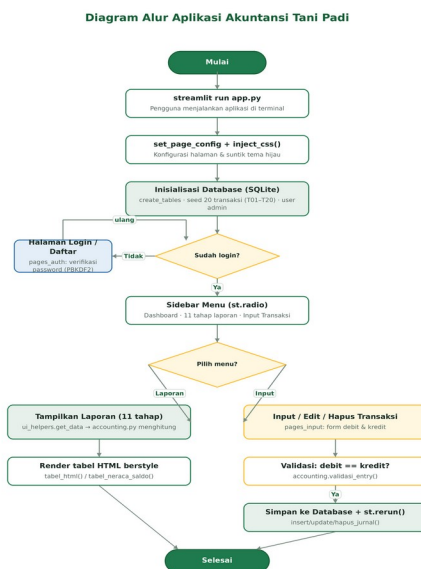


PENERAPAN SISTEM INFORMASI AKUNTANSI UNTUK PENCATATAN KEUANGAN USAHA TANI PADI (BUKU PANDUAN)



Aplikasi Akuntansi Tani Padi berbasis Streamlit

Disusun oleh:
Tim Pengembang Aplikasi
2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas limpahan rahmat dan karunia-Nya sehingga Buku Panduan Aplikasi Akuntansi Tani Padi ini dapat diselesaikan dengan baik. Buku panduan ini disusun sebagai dokumentasi resmi yang menjelaskan konsep, perancangan, hingga cara penggunaan aplikasi pencatatan keuangan untuk usaha tani padi.

Aplikasi Akuntansi Tani Padi dikembangkan untuk membantu petani dan pelaku usaha tani dalam mencatat transaksi keuangan secara rapi dan sistematis, mulai dari pencatatan jurnal umum hingga penyusunan laporan keuangan lengkap. Dengan antarmuka yang sederhana berbasis web, aplikasi ini diharapkan dapat menjembatani kebutuhan pencatatan keuangan yang akurat tanpa memerlukan latar belakang akuntansi yang mendalam.

Buku panduan ini memuat penjelasan menyeluruh, mulai dari latar belakang, arsitektur aplikasi, alur kerja sistem, struktur basis data, implementasi kode program, hingga langkah penggunaan setiap fitur yang dilengkapi dengan tangkapan layar aplikasi. Penjelasan kode disertai potongan program yang sesungguhnya agar pembaca dapat memahami logika di balik setiap fungsi.

Kami menyadari bahwa buku panduan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan demi penyempurnaan di masa mendatang. Semoga buku panduan ini bermanfaat bagi semua pihak yang menggunakannya.

Tim Pengembang Aplikasi

DAFTAR ISI

Klik kanan lalu pilih “Update Field” untuk memperbarui daftar isi.

BAB I PENDAHULUAN

1.1 Latar Belakang

Usaha tani padi merupakan salah satu sektor penting dalam perekonomian, namun pencatatan keuangannya sering kali masih dilakukan secara manual atau bahkan tidak dicatat sama sekali. Akibatnya, petani kesulitan mengetahui secara pasti besarnya pendapatan, beban, serta laba yang diperoleh dari setiap musim tanam. Ketidakteraturan pencatatan ini menyulitkan pengambilan keputusan dan evaluasi usaha.

Sistem informasi akuntansi hadir sebagai solusi untuk mengelola data keuangan secara terstruktur. Dengan memanfaatkan teknologi, proses pencatatan transaksi, penggolongan akun, hingga penyusunan laporan keuangan dapat dilakukan secara otomatis dan akurat. Aplikasi Akuntansi Tani Padi dibangun untuk menjawab kebutuhan tersebut dengan menyediakan siklus akuntansi lengkap dalam satu aplikasi berbasis web yang ringan dan mudah digunakan.

1.2 Tujuan

1. Menyediakan sarana pencatatan transaksi keuangan usaha tani padi yang rapi, akurat, dan mudah digunakan.
2. Menerapkan siklus akuntansi lengkap, mulai dari jurnal umum hingga laporan keuangan, secara otomatis.
3. Membantu pelaku usaha tani memahami posisi keuangan usahanya melalui laporan laba rugi, perubahan ekuitas, posisi keuangan, dan arus kas.
4. Menyediakan dokumentasi teknis yang menjelaskan arsitektur, kode program, dan cara penggunaan aplikasi secara menyeluruh.

1.3 Manfaat

- **Bagi pengguna:** Pencatatan keuangan menjadi lebih tertib dan dapat dipertanggungjawabkan.
- **Bagi usaha tani:** Laporan keuangan dihasilkan secara otomatis sehingga menghemat waktu dan mengurangi kesalahan hitung.

- **Bagi pengembang:** Menjadi acuan pengembangan dan pemeliharaan aplikasi pada masa mendatang.

1.4 Ruang Lingkup

Buku panduan ini mencakup penjelasan mengenai teknologi yang digunakan, alur kerja aplikasi, struktur basis data, implementasi kode program inti, serta panduan penggunaan setiap menu. Pembahasan difokuskan pada aplikasi Akuntansi Tani Padi yang dibangun menggunakan bahasa pemrograman Python dengan kerangka kerja Streamlit dan basis data SQLite.

BAB II PEMBAHASAN

2.1 Gambaran Umum Aplikasi

Aplikasi Akuntansi Tani Padi adalah sistem informasi akuntansi berbasis web yang menerapkan siklus akuntansi secara menyeluruh untuk usaha tani padi. Aplikasi dijalankan melalui peramban (browser) dan menampilkan data dalam bentuk tabel serta laporan keuangan yang mudah dibaca. Seluruh data transaksi disimpan secara permanen pada basis data lokal.

2.1.1 Teknologi dan Pustaka yang Digunakan

Aplikasi ini dibangun menggunakan beberapa teknologi dan pustaka utama sebagai berikut:

Teknologi / Pustaka	Keterangan
Python 3	Bahasa pemrograman utama yang digunakan untuk seluruh logika aplikasi.
Streamlit	Kerangka kerja antarmuka web untuk membangun halaman aplikasi secara cepat menggunakan Python murni.
SQLite	Basis data lokal berbasis berkas (tani_padi.db) untuk menyimpan data pengguna dan jurnal.
Pandas	Pustaka pengolahan data tabular yang digunakan untuk menyusun dan menampilkan tabel laporan.
Decimal	Modul bawaan Python untuk perhitungan uang dengan presisi tinggi sehingga tidak terjadi galat pembulatan.
hashlib	Modul bawaan Python untuk mengamankan kata sandi pengguna dengan algoritme PBKDF2-HMAC-SHA256.

2.1.2 Fitur Utama

- Autentikasi pengguna (login dan pendaftaran) dengan kata sandi terenkripsi.
- Pencatatan transaksi dengan sistem berpasangan (debit dan kredit) yang divalidasi otomatis.
- Penyajian siklus akuntansi lengkap dalam 11 tahap laporan.
- Dasbor ringkasan keuangan (pendapatan, beban, laba bersih, dan kas).
- Fitur ubah dan hapus transaksi yang fleksibel.

2.1.3 Siklus Akuntansi 11 Tahap

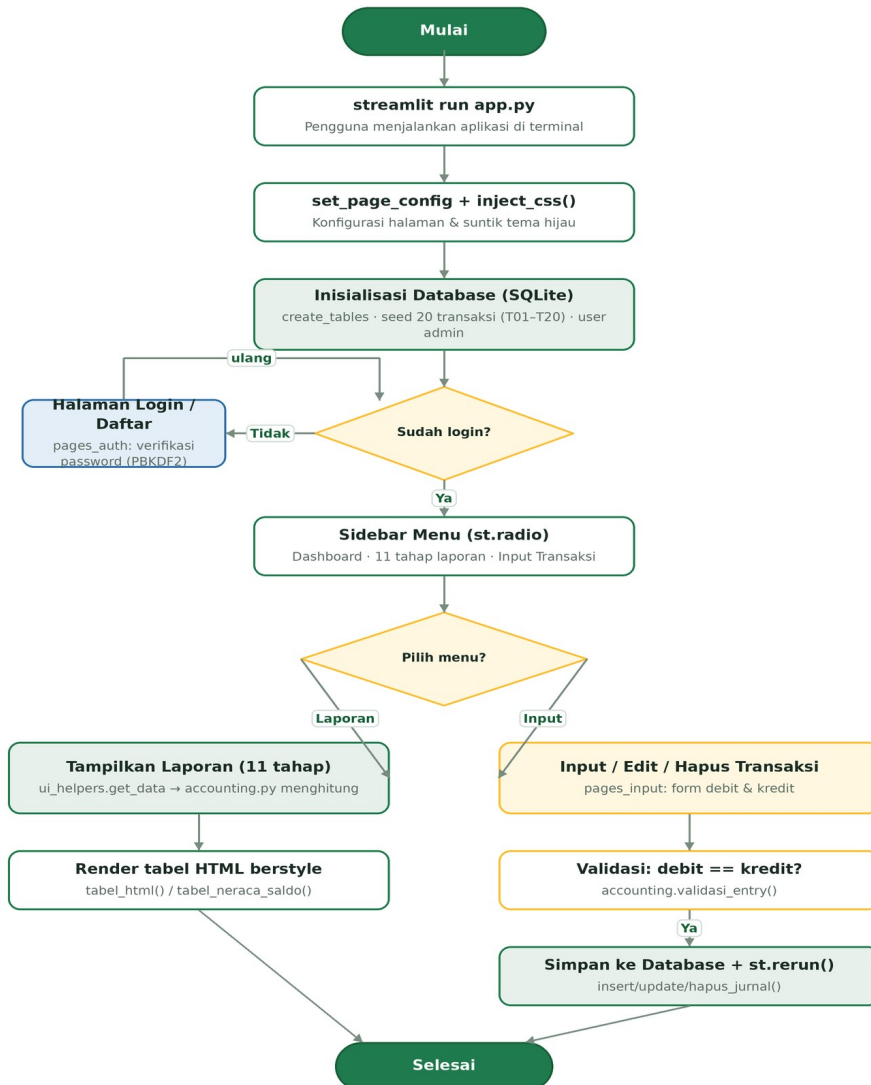
Aplikasi menyajikan keseluruhan siklus akuntansi yang terdiri atas sebelas tahap laporan berikut:

5. Jurnal Umum
6. Buku Besar
7. Neraca Saldo
8. Jurnal Penyesuaian
9. Neraca Saldo Setelah Penyesuaian
10. Laporan Laba Rugi
11. Laporan Perubahan Ekuitas
12. Laporan Posisi Keuangan (Neraca)
13. Laporan Arus Kas
14. Jurnal Penutup
15. Neraca Saldo Setelah Penutupan

2.2 Alur Kerja Aplikasi (Flowchart)

Diagram alur berikut menggambarkan jalannya aplikasi sejak dijalankan hingga pengguna dapat mengakses laporan dan mengelola transaksi. Ketika aplikasi dijalankan, konfigurasi halaman dan basis data disiapkan terlebih dahulu. Pengguna kemudian diarahkan ke halaman login. Setelah berhasil masuk, pengguna dapat memilih menu untuk melihat laporan atau mengelola transaksi.

Diagram Alur Aplikasi Akuntansi Tani Padi



Gambar 1. Diagram alur kerja aplikasi Akuntansi Tani Padi.

Inti dari alur ini adalah pemeriksaan status login dan validasi transaksi. Setiap transaksi yang diinput wajib memenuhi syarat keseimbangan, yaitu total debit harus sama dengan total kredit, sebelum data disimpan ke basis data.

2.3 Persiapan Lingkungan di Windows

Bagian ini menjelaskan cara menyiapkan dan menjalankan aplikasi pada komputer ber-sistem operasi Windows, langkah demi langkah, mulai dari memasang Python

sampai aplikasi terbuka di peramban. Urutan langkahnya adalah: (1) memasang Python dan mencentang “Add Python to PATH”, (2) membuat virtual environment, (3) mengaktifkan virtual environment, (4) memasang pustaka yang dibutuhkan, dan (5) menjalankan aplikasi. Ikuti langkah ini secara berurutan agar tidak ada tahap yang terlewat. Semua perintah pada bagian ini dijalankan melalui Command Prompt (cmd) atau PowerShell bawaan Windows.

2.3.1 Memasang Python di Windows

Python adalah bahasa pemrograman yang menjalankan seluruh aplikasi, sehingga harus dipasang terlebih dahulu. Unduh pemasang (installer) Python untuk Windows dari situs resmi di alamat <https://www.python.org/downloads/> , lalu jalankan berkas pemasang yang telah diunduh. Pada jendela pertama pemasang, WAJIB mencentang kotak “Add Python to PATH” (Tambahkan Python ke PATH) terlebih dahulu sebelum menekan tombol “Install Now”. Centang ini membuat perintah python dan py dapat dikenali dari Command Prompt mana pun. Setelah pemasangan selesai, buka Command Prompt (ketik “cmd” pada menu Start), lalu ketik perintah verifikasi berikut untuk memastikan Python sudah terpasang dengan benar.



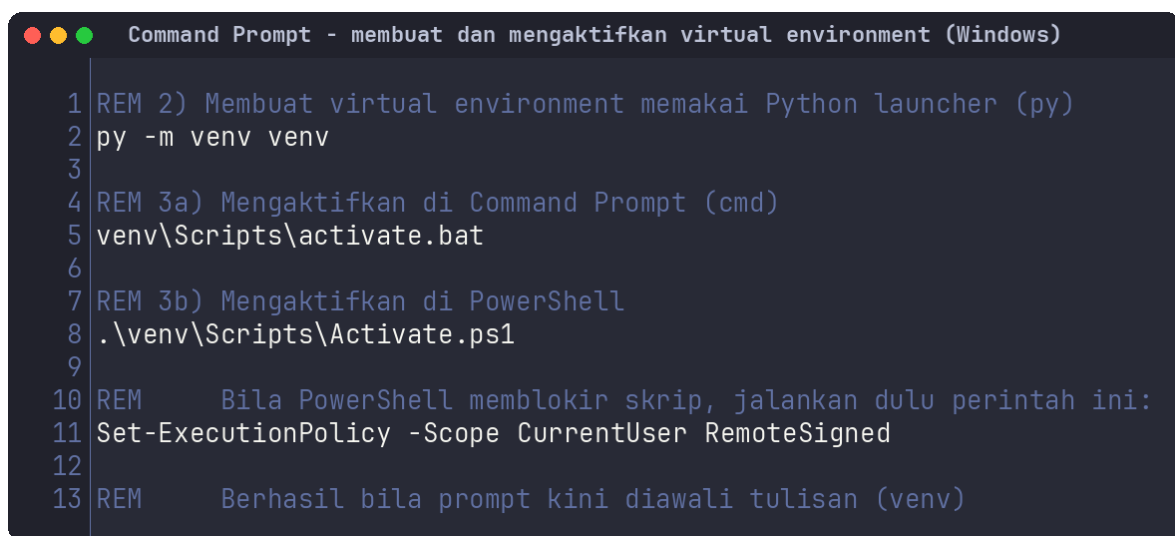
```
Command Prompt - memeriksa Python (Windows)
1 REM 1) Verifikasi Python sudah terpasang dan masuk ke PATH
2 REM (jalankan di Command Prompt / cmd)
3 py --version
4 python --version
```

Gambar 2. Memverifikasi pemasangan Python di Command Prompt Windows.

Apabila perintah `py --version` atau `python --version` menampilkan nomor versi (misalnya Python 3.12.x), berarti Python sudah berhasil dipasang dan siap digunakan. Bila muncul pesan bahwa perintah tidak dikenali, ulangi pemasangan dan pastikan kotak “Add Python to PATH” benar-benar tercentang.

2.3.2 Membuat dan Mengaktifkan Virtual Environment di Windows

Virtual environment adalah folder khusus yang menampung pustaka aplikasi secara terpisah, sehingga pustaka yang dipasang tidak bercampur atau mengganggu instalasi Python lain di komputer. Pada Windows, virtual environment dibuat menggunakan Python Launcher bawaan, yaitu perintah `py`, dengan menjalankan `py -m venv venv` (kata `venv` yang terakhir adalah nama folder yang akan dibuat). Setelah folder terbentuk, virtual environment perlu diaktifkan agar perintah `pip` dan `streamlit` dijalankan di dalam lingkungan tersebut. Cara mengaktifkannya berbeda sedikit tergantung jenis terminal yang dipakai, seperti terlihat pada gambar berikut.



```
1 REM 2) Membuat virtual environment memakai Python launcher (py)
2 py -m venv venv
3
4 REM 3a) Mengaktifkan di Command Prompt (cmd)
5 venv\Scripts\activate.bat
6
7 REM 3b) Mengaktifkan di PowerShell
8 .\venv\Scripts\Activate.ps1
9
10 REM     Bila PowerShell memblokir skrip, jalankan dulu perintah ini:
11 Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
12
13 REM     Berhasil bila prompt kini diawali tulisan (venv)
```

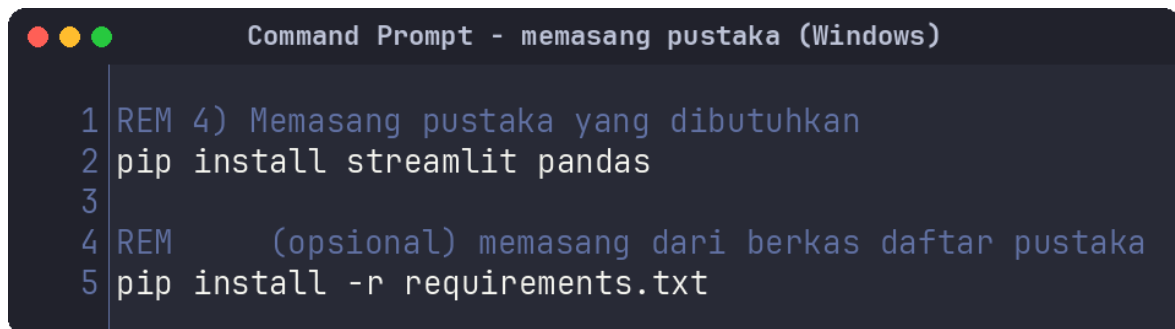
Gambar 3. Membuat dan mengaktifkan virtual environment di Windows.

- **Pada Command Prompt (cmd):** jalankan `venv\Scripts\activate.bat` (atau cukup `venv\Scripts\activate`).
- **Pada PowerShell:** jalankan `.\venv\Scripts\Activate.ps1`. Bila PowerShell menolak karena kebijakan keamanan, jalankan dulu perintah `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` lalu coba lagi.
- **Tanda berhasil:** virtual environment berhasil aktif bila baris perintah (prompt) kini diawali tulisan `(venv)`.

Perhatikan bahwa pemisah folder pada Windows memakai garis miring terbalik (backslash), yaitu `venv\Scripts\...`, berbeda dengan Linux atau macOS yang memakai garis miring biasa.

2.3.3 Memasang Pustaka yang Dibutuhkan

Setelah virtual environment aktif (prompt diawali (venv)), pustaka pendukung aplikasi dipasang menggunakan perintah pip, yaitu pengelola paket bawaan Python. Dua pustaka utama yang dibutuhkan adalah Streamlit (untuk antarmuka web) dan Pandas (untuk mengolah tabel). Pemasangan dapat dilakukan langsung dengan menyebut nama pustakanya, atau melalui berkas daftar pustaka requirements.txt bila tersedia. Pastikan komputer terhubung ke internet karena pip akan mengunduh pustaka tersebut.

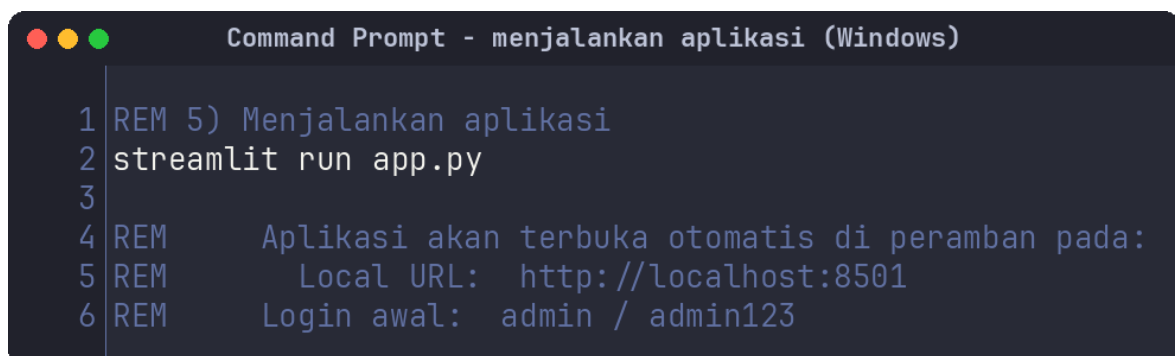


```
1 REM 4) Memasang pustaka yang dibutuhkan
2 pip install streamlit pandas
3
4 REM (opsional) memasang dari berkas daftar pustaka
5 pip install -r requirements.txt
```

Gambar 4. Memasang pustaka Streamlit dan Pandas melalui pip di Windows.

2.3.4 Menjalankan Aplikasi

Bila seluruh pustaka sudah terpasang, aplikasi dijalankan dengan perintah streamlit run app.py dari dalam folder proyek. Streamlit kemudian menyalakan server lokal dan secara otomatis membuka aplikasi pada peramban (browser) di alamat <http://localhost:8501>. Pengguna dapat masuk menggunakan akun demo admin dengan kata sandi admin123. Selama aplikasi berjalan, jendela Command Prompt harus tetap dibuka; menutupnya akan menghentikan aplikasi.

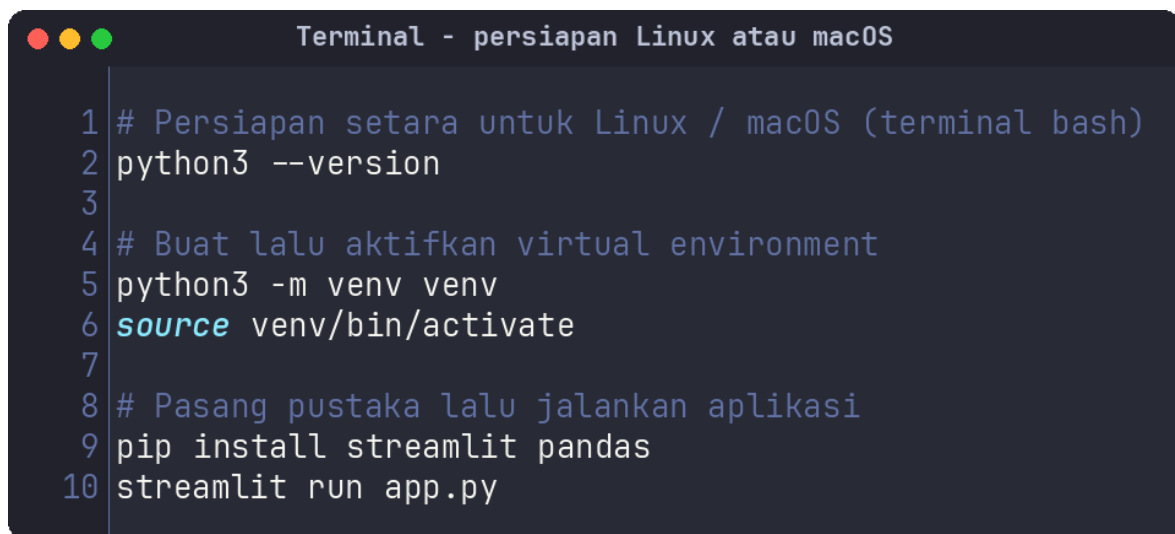


```
1 REM 5) Menjalankan aplikasi
2 streamlit run app.py
3
4 REM Aplikasi akan terbuka otomatis di peramban pada:
5 REM Local URL: http://localhost:8501
6 REM Login awal: admin / admin123
```

Gambar 5. Menjalankan aplikasi dengan Streamlit di Windows.

2.3.5 Alternatif untuk Linux/macOS

Bagi pengguna sistem operasi Linux atau macOS, langkahnya serupa namun perintahnya sedikit berbeda: pembuatan virtual environment memakai `python3 -m venv venv` dan pengaktifannya memakai `source venv/bin/activate` (dengan garis miring biasa). Setelah aktif, pemasangan pustaka dan menjalankan aplikasi menggunakan perintah yang sama, yaitu `pip install streamlit pandas` dan `streamlit run app.py`.



```
1 # Persiapan setara untuk Linux / macOS (terminal bash)
2 python3 --version
3
4 # Buat lalu aktifkan virtual environment
5 python3 -m venv venv
6 source venv/bin/activate
7
8 # Pasang pustaka lalu jalankan aplikasi
9 pip install streamlit pandas
10 streamlit run app.py
```

Gambar 6. Persiapan lingkungan pada terminal Linux atau macOS (alternatif).

2.4 Struktur Basis Data dan Bagan Akun

Aplikasi menggunakan basis data SQLite dengan tiga tabel utama yang saling berelasi. Struktur ini dirancang untuk mendukung pencatatan jurnal berpasangan, di mana setiap jurnal dapat memiliki beberapa baris akun (debit dan kredit).

2.4.1 Skema Tabel

Skema tabel dibuat melalui fungsi `create_tables()`. Terdapat tiga tabel: `users` untuk menyimpan data pengguna, `jurnal` sebagai kepala (header) transaksi, dan `jurnal_baris` sebagai rincian akun pada setiap transaksi.

```
database.py - skema tabel

1 def create_tables(conn: sqlite3.Connection) -> None:
2     cur = conn.cursor()
3     cur.execute(
4         """
5         CREATE TABLE IF NOT EXISTS users (
6             id            INTEGER PRIMARY KEY AUTOINCREMENT,
7             username      TEXT UNIQUE NOT NULL,
8             password_hash TEXT NOT NULL,
9             nama          TEXT
10        )
11        """
12    )
13    cur.execute(
14        """
15        CREATE TABLE IF NOT EXISTS jurnal (
16            id            INTEGER PRIMARY KEY AUTOINCREMENT,
17            kode          TEXT NOT NULL,
18            tanggal      TEXT NOT NULL,
19            keterangan     TEXT,
20            tipe          TEXT NOT NULL DEFAULT 'umum'
21        )
22        """
23    )
24    cur.execute(
25        """
26        CREATE TABLE IF NOT EXISTS jurnal_baris (
27            id            INTEGER PRIMARY KEY AUTOINCREMENT,
28            jurnal_id     INTEGER NOT NULL,
29            kode_akun     TEXT NOT NULL,
30            akun          TEXT NOT NULL,
31            debit         TEXT NOT NULL DEFAULT '0',
32            kredit        TEXT NOT NULL DEFAULT '0',
33            FOREIGN KEY (jurnal_id) REFERENCES jurnal(id) ON DELETE CASCADE
34        )
35        """
36    )
37    conn.commit()
```

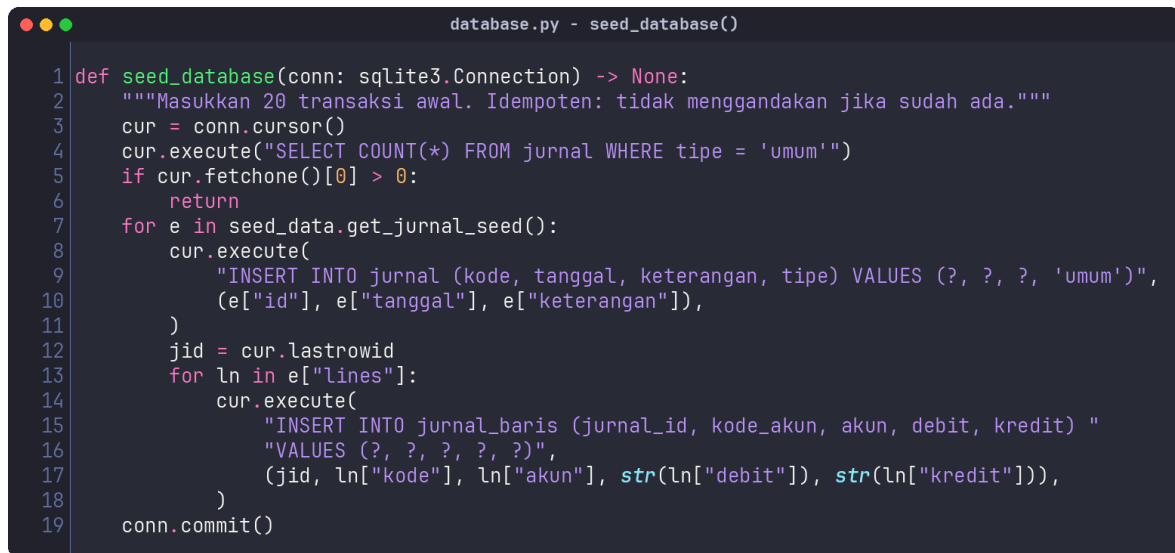
Gambar 7. Definisi skema tabel pada database.py.

- **users:** menyimpan data akun pengguna meliputi username, kata sandi terenkripsi, dan nama.
- **jurnal:** menyimpan kepala transaksi berisi kode, tanggal, keterangan, dan tipe jurnal.
- **jurnal_baris:** menyimpan rincian baris akun (kode akun, nama akun, debit, kredit) yang merujuk ke tabel jurnal melalui jurnal_id.

Nilai uang (debit dan kredit) disimpan dalam tipe TEXT agar presisi angka tetap terjaga ketika diolah kembali menggunakan tipe Decimal di sisi aplikasi.

2.4.2 Pengisian Data Awal (Seed)

Saat aplikasi pertama kali dijalankan, basis data diisi dengan data awal berupa 20 transaksi contoh (T01 sampai T20) dan satu akun pengguna admin. Hal ini memudahkan pengguna baru untuk langsung melihat contoh laporan yang telah terisi.



```
1 def seed_database(conn: sqlite3.Connection) -> None:
2     """Masukkan 20 transaksi awal. Idempoten: tidak menggandakan jika sudah ada."""
3     cur = conn.cursor()
4     cur.execute("SELECT COUNT(*) FROM jurnal WHERE tipe = 'umum'")
5     if cur.fetchone()[0] > 0:
6         return
7     for e in seed_data.get_jurnal_seed():
8         cur.execute(
9             "INSERT INTO jurnal (kode, tanggal, keterangan, tipe) VALUES (?, ?, ?, 'umum')",
10            (e["id"], e["tanggal"], e["keterangan"]),
11        )
12        jid = cur.lastrowid
13        for ln in e["lines"]:
14            cur.execute(
15                "INSERT INTO jurnal_baris (jurnal_id, kode_akun, akun, debit, kredit) "
16                "VALUES (?, ?, ?, ?, ?)",
17                (jid, ln["kode"], ln["akun"], str(ln["debit"]), str(ln["kredit"])),
18            )
19     conn.commit()
```

Gambar 8. Fungsi pengisian data awal basis data.

2.4.3 Bagan Akun (Chart of Accounts)

Aplikasi menggunakan bagan akun yang terdiri atas 15 akun, mencakup kelompok aset, ekuitas, pendapatan, dan beban. Bagan akun ini menjadi acuan dalam pencatatan setiap transaksi.

```

seed_data.py - bagan akun
1 CHART_OF_ACCOUNTS = [
2     {"kode": "111", "akun": "Kas", "tipe": "aset", "saldo_normal": "debit"},
3     {"kode": "112", "akun": "Piutang Usaha", "tipe": "aset", "saldo_normal": "debit"},
4     {"kode": "113", "akun": "Perlengkapan", "tipe": "aset", "saldo_normal": "debit"},
5     {"kode": "311", "akun": "Modal Petani", "tipe": "ekuitas", "saldo_normal": "kredit"},
6     {"kode": "312", "akun": "Prive", "tipe": "kontra_ekuitas", "saldo_normal": "debit"},
7     {"kode": "313", "akun": "Ikhtisar Laba Rugi", "tipe": "ikhtisar", "saldo_normal": "kredit"},
8     {"kode": "411", "akun": "Pendapatan Penjualan Gabah", "tipe": "pendapatan", "saldo_normal": "kredit"},
9     {"kode": "511", "akun": "Beban Benih", "tipe": "beban", "saldo_normal": "debit"},
10    {"kode": "512", "akun": "Beban Pupuk", "tipe": "beban", "saldo_normal": "debit"},
11    {"kode": "513", "akun": "Beban Tenaga Kerja", "tipe": "beban", "saldo_normal": "debit"},
12    {"kode": "514", "akun": "Beban Pestisida & Obat", "tipe": "beban", "saldo_normal": "debit"},
13    {"kode": "515", "akun": "Beban Angkut", "tipe": "beban", "saldo_normal": "debit"},
14    {"kode": "516", "akun": "Beban Konsumsi", "tipe": "beban", "saldo_normal": "debit"},
15    {"kode": "517", "akun": "Beban Administrasi", "tipe": "beban", "saldo_normal": "debit"},
16    {"kode": "518", "akun": "Beban Perlengkapan", "tipe": "beban", "saldo_normal": "debit"},
17 ]
18
19 # index cepat: kode → info akun, nama → info akun
20 AKUN_BY_KODE = {a["kode"]: a for a in CHART_OF_ACCOUNTS}
21 AKUN_BY_NAMA = {a["akun"]: a for a in CHART_OF_ACCOUNTS}
22
23
24 def get_chart_of_accounts():
25     """Kembalikan salinan bagan akun."""
26     return [dict(a) for a in CHART_OF_ACCOUNTS]

```

Gambar 9. Definisi bagan akun pada seed_data.py.

Kelompok Akun	Contoh Akun
Aset	Kas, Perlengkapan, Peralatan, Akumulasi Penyusutan Peralatan
Ekuitas	Modal Pemilik, Prive
Pendapatan	Pendapatan Penjualan Gabah
Beban	Beban Benih, Beban Pupuk, Beban Tenaga Kerja, Beban Pestisida & Obat, Beban Angkut, Beban Konsumsi, Beban Administrasi, Beban Perlengkapan

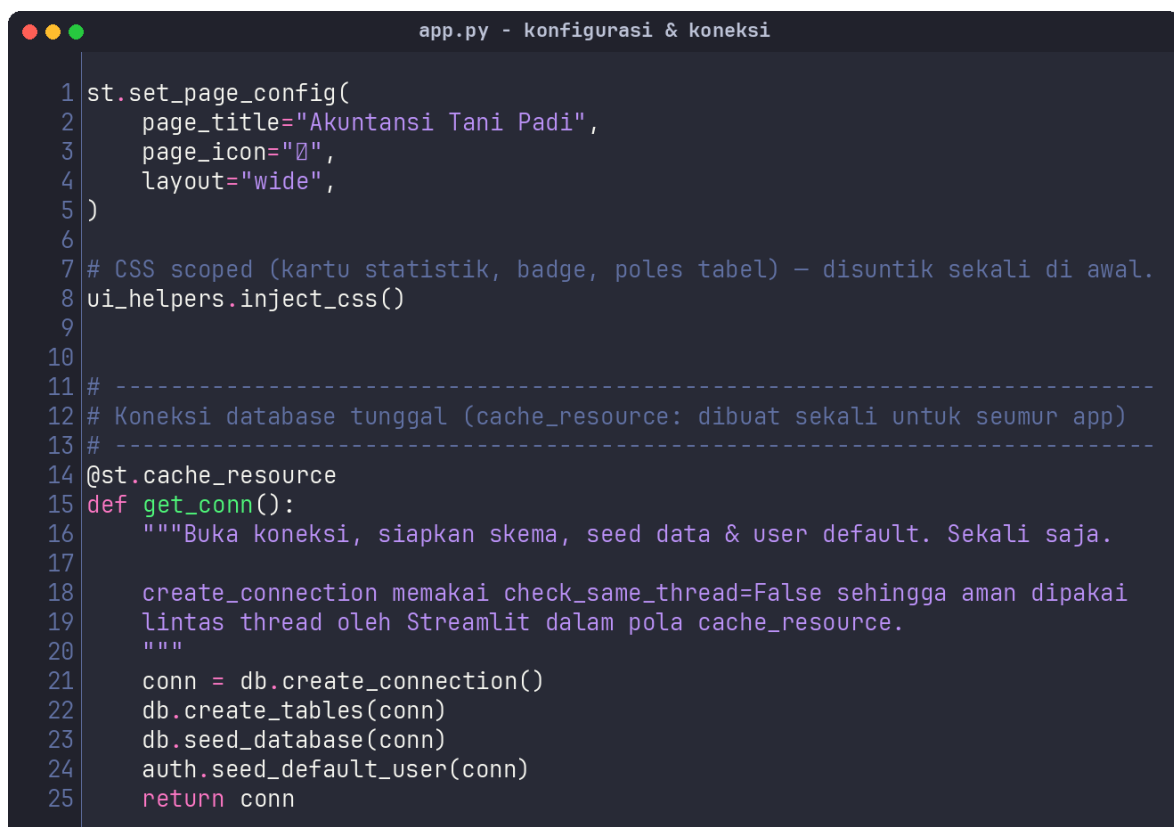
2.5 Implementasi Kode Program

Bagian ini menjelaskan implementasi kode program inti aplikasi. Setiap potongan kode disertai penjelasan mengenai fungsi dan logikanya agar mudah dipahami.

2.5.1 Konfigurasi dan Koneksi Aplikasi (app.py)

Berkas app.py merupakan titik awal (entry point) aplikasi, yaitu berkas pertama yang dijalankan ketika perintah streamlit run app.py dipanggil. Pada bagian ini dilakukan konfigurasi halaman melalui st.set_page_config untuk menentukan judul tab, ikon padi, dan tata letak lebar (wide), lalu gaya tampilan (CSS) bernuansa hijau

disuntikkan satu kali agar seluruh aplikasi tampak seragam dengan tema pertanian. Konfigurasi halaman ini wajib menjadi perintah Streamlit yang pertama dipanggil; bila tidak, Streamlit akan menolaknya. Selanjutnya koneksi ke basis data dibuka melalui fungsi `get_conn` yang ditandai `@st.cache_resource` (artinya koneksi dibuat sekali saja dan dipakai ulang selama aplikasi hidup, bukan dibuka berulang setiap interaksi). Di dalam `get_conn`, skema tabel disiapkan, data awal diisi, dan akun admin dibuat, sehingga aplikasi siap pakai sejak pertama kali dijalankan.



```
app.py - konfigurasi & koneksi
1 st.set_page_config(
2     page_title="Akuntansi Tani Padi",
3     page_icon="🌾",
4     layout="wide",
5 )
6
7 # CSS scoped (kartu statistik, badge, poles tabel) – disuntik sekali di awal.
8 ui_helpers.inject_css()
9
10
11 # -----
12 # Koneksi database tunggal (cache_resource: dibuat sekali untuk seumur app)
13 # -----
14 @st.cache_resource
15 def get_conn():
16     """Buka koneksi, siapkan skema, seed data & user default. Sekali saja.
17
18     create_connection memakai check_same_thread=False sehingga aman dipakai
19     lintas thread oleh Streamlit dalam pola cache_resource.
20     """
21     conn = db.create_connection()
22     db.create_tables(conn)
23     db.seed_database(conn)
24     auth.seed_default_user(conn)
25     return conn
```

Gambar 10. Konfigurasi halaman dan koneksi awal pada `app.py`.

Penjelasan:

- `st.set_page_config` menentukan judul “Akuntansi Tani Padi”, ikon padi, dan tata letak wide (lebar penuh).
- Pemanggilan `set_page_config` harus berada paling awal sebelum perintah Streamlit lain agar tidak menimbulkan galat.
- `ui_helpers.inject_css()` menyuntikkan gaya CSS hijau satu kali untuk kartu statistik, badge, dan tabel.

- Fungsi `get_conn` diberi penanda `@st.cache_resource` (cache sumber daya) agar koneksi basis data hanya dibuat sekali dan dipakai ulang.
- Di dalam `get_conn` dipanggil `create_tables`, `seed_database`, dan `seed_default_user` sehingga tabel, data contoh, dan akun admin langsung tersedia.
- Koneksi memakai `check_same_thread=False` supaya aman digunakan lintas thread (proses paralel) oleh Streamlit.
- Modul ini sengaja tidak memuat logika akuntansi; tugasnya murni mengatur konfigurasi, koneksi, dan pengalihan halaman.

Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.5.2 Menu dan Pengalihan Halaman (`app.py`)

Setelah pengguna berhasil masuk, aplikasi menampilkan menu navigasi pada sidebar (panel samping) menggunakan `st.radio`, yaitu daftar pilihan berbentuk tombol pilih tunggal. Daftar menu disimpan dalam variabel `MENU` yang berisi urutan tetap mulai dari Dashboard, sebelas tahap laporan akuntansi, hingga menu Input Transaksi. Fungsi `_render_sidebar` bertugas menampilkan sapaan nama pengguna, tombol keluar (logout), serta menu tersebut, lalu mengembalikan pilihan yang ditekan pengguna. Pilihan itu kemudian diteruskan ke fungsi `_dispatch` yang mencocokkan setiap pilihan menu dengan fungsi penampil halaman yang sesuai. Dengan pola terpusat seperti ini, penambahan atau perubahan menu cukup dilakukan di satu tempat sehingga navigasi tetap rapi dan mudah dirawat.

```
app.py - menu & dispatch

1 MENU = [
2     "Dashboard",
3     "1. Jurnal Umum",
4     "2. Buku Besar",
5     "3. Neraca Saldo",
6     "4. Jurnal Penyesuaian",
7     "5. NS Setelah Penyesuaian",
8     "6. Laporan Laba Rugi",
9     "7. Perubahan Ekuitas",
10    "8. Posisi Keuangan",
11    "9. Arus Kas",
12    "10. Jurnal Penutup",
13    "11. NS Setelah Penutupan",
14    "Input Transaksi",
15 ]
16
17 # ...
18
19 def _dispatch(pilihan, conn, data):
20     """Arahkan pilihan menu ke fungsi render halaman yang sesuai.
21
22     Halaman laporan menerima `data` (hasil ui_helpers.get_data); halaman
23     Dashboard & Input menerima `conn`.
24     """
25     if pilihan == "Dashboard":
26         pages_dashboard.render_dashboard(conn, data)
27     elif pilihan == "1. Jurnal Umum":
28         pages_reports_a.render_jurnal_umum(data)
29     elif pilihan == "2. Buku Besar":
30         pages_reports_a.render_buku_besar(data)
31     elif pilihan == "3. Neraca Saldo":
32         pages_reports_a.render_neraca_saldo(data)
33     elif pilihan == "4. Jurnal Penyesuaian":
34         pages_reports_b.render_jurnal_penyesuaian(data)
35     elif pilihan == "5. NS Setelah Penyesuaian":
36         pages_reports_b.render_ns_penyesuaian(data)
37     elif pilihan == "6. Laporan Laba Rugi":
38         pages_reports_b.render_laba_rugi(data)
39     elif pilihan == "7. Perubahan Ekuitas":
40         pages_reports_c.render_ekuitas(data)
41     elif pilihan == "8. Posisi Keuangan":
42         pages_reports_c.render_posisi_keuangan(data)
43     elif pilihan == "9. Arus Kas":
44         pages_reports_c.render_ arus_kas(data)
45     elif pilihan == "10. Jurnal Penutup":
46         pages_reports_d.render_jurnal_penutup(data)
47     elif pilihan == "11. NS Setelah Penutupan":
48         pages_reports_d.render_ns_penutupan(data)
49     elif pilihan == "Input Transaksi":
50         pages_input.render_input(conn)
```

Gambar 11. Logika menu dan pengalihan halaman pada app.py.

Penjelasan:

- Daftar MENU memuat urutan tetap: Dashboard, 11 laporan siklus akuntansi, dan Input Transaksi.
- `st.radio` menampilkan menu sebagai pilihan tunggal di sidebar (panel samping kiri).
- Nama pengguna pada sapaan dilewatkan `html.escape()` agar aman dari penyisipan kode berbahaya (XSS).
- Tombol “Keluar” memanggil `st.session_state.clear()` lalu `st.rerun()` untuk mengakhiri sesi login.
- Fungsi `_dispatch` memetakan tiap teks pilihan menu ke satu fungsi render halaman tertentu.
- Halaman laporan menerima data hasil olahan, sedangkan Dashboard dan Input menerima koneksi (`conn`) langsung.
- Pemusatan logika menu di satu fungsi memudahkan penambahan menu baru tanpa mengubah banyak berkas.

Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.5.3 Alur Utama Aplikasi (`app.py`)

Fungsi `main()` adalah pengatur keseluruhan alur aplikasi dan dipanggil di bagian paling bawah berkas `app.py`. Pertama, ia memanggil `get_conn` untuk memperoleh koneksi basis data yang sudah disiapkan. Kemudian ia memeriksa status login pengguna yang disimpan pada `session_state` (memori sesi yang bertahan selama pengguna membuka aplikasi). Bila pengguna belum masuk, aplikasi hanya menampilkan halaman autentikasi lalu berhenti dengan `st.stop()` sehingga menu dan laporan tidak bisa diakses tanpa login. Sebaliknya, bila sudah masuk, aplikasi menampilkan sidebar, menyiapkan data laporan melalui `ui_helpers.get_data`, lalu menampilkan halaman sesuai pilihan menu. Pola ini menjadi “gerbang” keamanan yang memastikan hanya pengguna terverifikasi yang dapat melihat data keuangan.

```
app.py - alur utama main()

1 def main():
2     conn = get_conn()
3
4     # Inisialisasi state login.
5     if "logged_in" not in st.session_state:
6         st.session_state.logged_in = False
7
8     # Gerbang login: belum masuk → hanya halaman auth, lalu berhenti.
9     if not st.session_state.get("logged_in"):
10         pages_auth.render_auth(conn)
11         st.stop()
12
13     # Sudah login: sidebar + halaman terpilih.
14     pilihan = _render_sidebar()
15     data = ui_helpers.get_data(conn)
16     _dispatch(pilihan, conn, data)
17
18
19 main()
```

Gambar 12. Fungsi utama main() yang mengatur alur aplikasi.

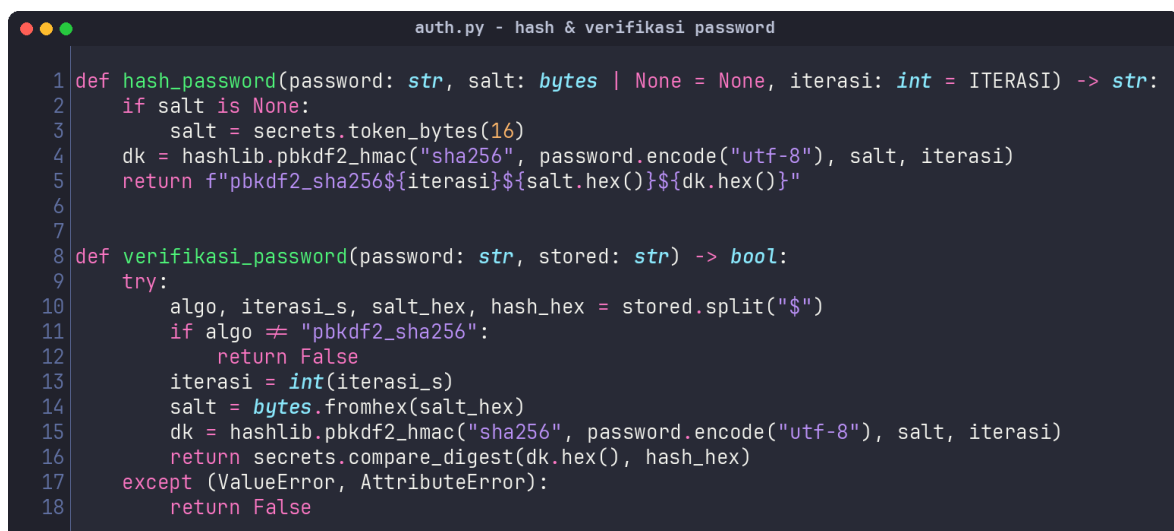
Penjelasan:

- main() pertama-tama memanggil get_conn() untuk memperoleh koneksi basis data yang siap pakai.
- Status login disimpan pada st.session_state.logged_in dan diinisialisasi False bila belum ada.
- Bila belum login, hanya pages_auth.render_auth yang ditampilkan, diakhiri st.stop() agar eksekusi berhenti.
- st.stop() berfungsi sebagai gerbang keamanan: menu dan laporan tidak dimuat sebelum pengguna masuk.
- Setelah login berhasil, _render_sidebar dipanggil untuk memperoleh pilihan menu pengguna.
- ui_helpers.get_data(conn) menyiapkan seluruh data laporan sebelum halaman dirender.
- Pemisahan tegas antara keadaan “belum login” dan “sudah login” membuat alur mudah dipahami dan diaudit.

Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.5.4 Pengamanan Kata Sandi (auth.py)

Berkas auth.py menangani keamanan akun pengguna. Kata sandi tidak pernah disimpan dalam bentuk aslinya, melainkan diubah menjadi hash (sidik jari digital satu arah yang tidak bisa dikembalikan ke teks asli) menggunakan algoritme PBKDF2-HMAC-SHA256. Fungsi `hash_password` membangkitkan salt acak (potongan data acak unik per pengguna) sepanjang 16 byte, lalu mengolah kata sandi sebanyak 200.000 iterasi (pengulangan) agar sulit ditebak walau dengan komputer cepat. Hasilnya disimpan dalam satu teks berformat `pbkdf2_sha256$<iterasi>$<salt>$<hash>` sehingga seluruh informasi verifikasi tersimpan rapi tanpa membocorkan kata sandi. Saat login, `verifikasi_password` memproses kata sandi yang dimasukkan dengan salt dan iterasi yang sama, lalu membandingkan hasilnya memakai `secrets.compare_digest` yang aman terhadap serangan pengukuran waktu (timing attack).



```
auth.py - hash & verifikasi password
1 def hash_password(password: str, salt: bytes | None = None, iterasi: int = ITERASI) -> str:
2     if salt is None:
3         salt = secrets.token_bytes(16)
4     dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, iterasi)
5     return f"pbkdf2_sha256${iterasi}${salt.hex()}${dk.hex()}"
6
7
8 def verifikasi_password(password: str, stored: str) -> bool:
9     try:
10         algo, iterasi_s, salt_hex, hash_hex = stored.split("$")
11         if algo != "pbkdf2_sha256":
12             return False
13         iterasi = int(iterasi_s)
14         salt = bytes.fromhex(salt_hex)
15         dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, iterasi)
16         return secrets.compare_digest(dk.hex(), hash_hex)
17     except (ValueError, AttributeError):
18         return False
```

Gambar 13. Fungsi hash dan verifikasi kata sandi pada auth.py.

Penjelasan:

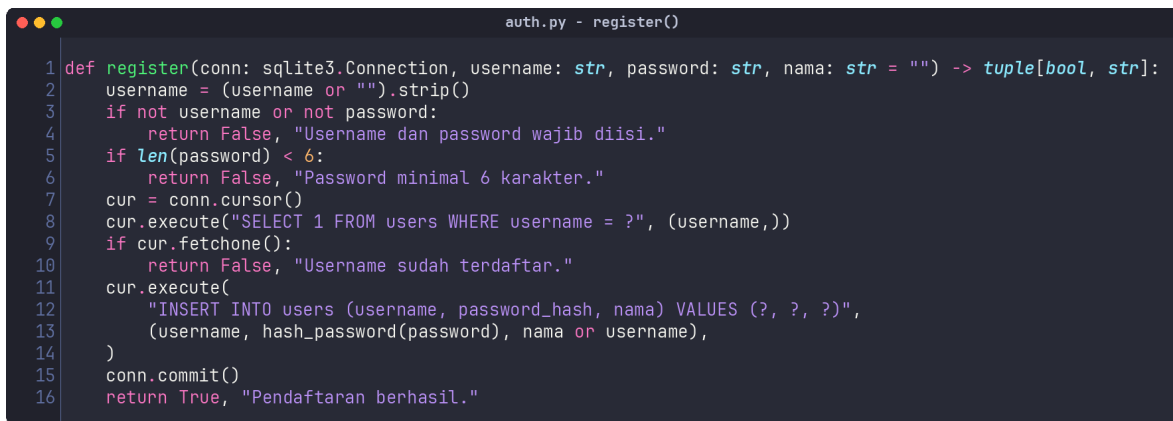
- Konstanta ITERASI bernilai 200.000, menentukan berapa kali kata sandi diolah agar hash sulit dibongkar.

- `secrets.token_bytes(16)` menghasilkan salt acak unik untuk tiap pengguna sehingga dua sandi sama tetap berbeda hash-nya.
- `hashlib.pbkdf2_hmac("sha256", ...)` adalah inti perhitungan hash satu arah berbasis SHA-256.
- Hasil disimpan sebagai teks `pbkdf2_sha256$<iterasi>$<salt>$<hash>` agar mudah dipisah kembali saat verifikasi.
- `verifikasi_password` memecah teks tersimpan, menghitung ulang hash dari kata sandi masukan, lalu membandingkannya.
- Perbandingan memakai `secrets.compare_digest` yang aman dari timing attack (penebak lewat selisih waktu proses).
- Pendekatan ini memakai pustaka bawaan Python (`hashlib` + `secrets`) tanpa dependensi tambahan, sehingga ringan dan andal.

Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.5.5 Pendaftaran Pengguna (`auth.py`)

Fungsi `register` bertugas menambahkan pengguna baru ke basis data dengan beberapa lapis pemeriksaan agar data tetap bersih. Mula-mula `username` dirapikan dari spasi berlebih, lalu dipastikan `username` dan kata sandi tidak kosong. Kata sandi juga diwajibkan minimal enam karakter sebagai syarat keamanan dasar. Sebelum menyimpan, fungsi memeriksa apakah `username` sudah dipakai; bila sudah ada, pendaftaran ditolak agar tidak terjadi duplikasi. Bila semua syarat terpenuhi, kata sandi dienkripsi memakai `hash_password`, kemudian data dimasukkan ke tabel `users` dan disimpan permanen. Fungsi mengembalikan pasangan nilai (berhasil/gagal beserta pesannya) sehingga halaman pendaftaran dapat menampilkan umpan balik yang jelas kepada pengguna.



```

auth.py - register()
1 def register(conn: sqlite3.Connection, username: str, password: str, nama: str = "") -> tuple[bool, str]:
2     username = (username or "").strip()
3     if not username or not password:
4         return False, "Username dan password wajib diisi."
5     if len(password) < 6:
6         return False, "Password minimal 6 karakter."
7     cur = conn.cursor()
8     cur.execute("SELECT 1 FROM users WHERE username = ?", (username,))
9     if cur.fetchone():
10        return False, "Username sudah terdaftar."
11    cur.execute(
12        "INSERT INTO users (username, password_hash, nama) VALUES (?, ?, ?)",
13        (username, hash_password(password), nama or username),
14    )
15    conn.commit()
16    return True, "Pendaftaran berhasil."

```

Gambar 14. Fungsi pendaftaran pengguna baru pada auth.py.

Penjelasan:

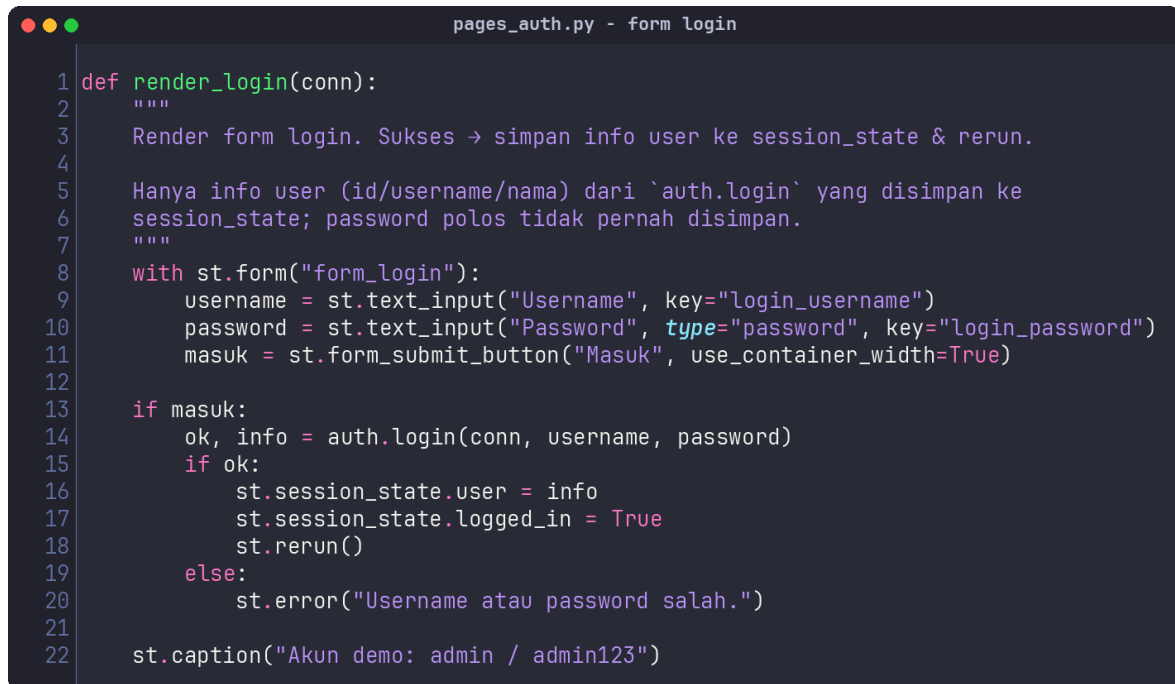
- Username dirapikan dengan `.strip()` untuk menghapus spasi di awal dan akhir.
- Username dan kata sandi wajib diisi; bila kosong, pendaftaran langsung ditolak dengan pesan jelas.
- Kata sandi diwajibkan minimal 6 karakter sebagai pengaman dasar.
- Perintah `SELECT 1 FROM users WHERE username = ?` memeriksa apakah username sudah terpakai sebelum menyimpan.
- Tanda tanya (?) pada perintah SQL adalah parameter aman yang mencegah serangan SQL injection (penyisipan perintah jahat).
- Kata sandi disimpan sebagai hash hasil `hash_password`, bukan teks asli.
- Fungsi mengembalikan tuple (bool, pesan) agar antarmuka tahu pendaftaran berhasil atau gagal beserta alasannya.

Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.5.6 Formulir Login (pages_auth.py)

Berkas `pages_auth.py` menyajikan halaman masuk dan daftar dalam dua tab ("Masuk" dan "Daftar") di kolom tengah layar. Fungsi `render_login` menampilkan formulir berisi kolom username dan kata sandi, dengan kolom kata sandi disembunyikan (`type="password"`). Modul ini bersifat read-only terhadap backend, artinya ia tidak menghitung hash sendiri melainkan hanya memanggil `auth.login`

untuk memverifikasi kredensial. Apabila login berhasil, informasi pengguna (id, username, nama) disimpan ke session_state dan tanda logged_in dijadikan True, lalu halaman dimuat ulang dengan st.rerun. Bila gagal, pesan kesalahan ditampilkan tanpa membocorkan apakah username atau kata sandinya yang salah, sehingga lebih aman.



```
1 def render_login(conn):
2     """
3     Render form login. Sukses → simpan info user ke session_state & rerun.
4
5     Hanya info user (id/username/nama) dari `auth.login` yang disimpan ke
6     session_state; password polos tidak pernah disimpan.
7     """
8     with st.form("form_login"):
9         username = st.text_input("Username", key="login_username")
10        password = st.text_input("Password", type="password", key="login_password")
11        masuk = st.form_submit_button("Masuk", use_container_width=True)
12
13    if masuk:
14        ok, info = auth.login(conn, username, password)
15        if ok:
16            st.session_state.user = info
17            st.session_state.logged_in = True
18            st.rerun()
19        else:
20            st.error("Username atau password salah.")
21
22    st.caption("Akun demo: admin / admin123")
```

Gambar 15. Formulir login pada pages_auth.py.

Penjelasan:

- `st.form("form_login")` membungkus kolom input agar diproses sekaligus saat tombol ditekan.
- Kolom kata sandi memakai `type="password"` sehingga karakter tersembunyi saat diketik.
- Verifikasi kredensial didelegasikan ke `auth.login`; halaman tidak pernah menghitung hash sendiri.
- Jika berhasil, info pengguna disimpan ke `st.session_state.user` dan `logged_in` di-set True.
- `st.rerun()` memuat ulang halaman agar tampilan langsung berpindah ke menu utama.
- Pesan kesalahan dibuat umum ("Username atau password salah") agar tidak membocorkan akun mana yang ada.

- Hanya data non-sensitif (id/username/nama) yang disimpan; kata sandi polos tidak pernah ikut tersimpan.

Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.5.7 Penomoran Kode dan Penyimpanan Jurnal (database.py)

Bagian ini berisi dua fungsi yang bekerja sama saat sebuah transaksi baru disimpan. Fungsi `next_kode` menghitung jumlah jurnal umum yang sudah ada lalu menambah satu, menghasilkan kode urut berformat `T##` (misalnya `T21` setelah `T20`). Fungsi `insert_jurnal` menyimpan transaksi tersebut, tetapi sebelum menulis apa pun ia memanggil `acc.validasi_entry` untuk memastikan total debit sama dengan total kredit; bila tidak seimbang, fungsi melempar `ValueError` sehingga tidak ada data yang tersimpan. Setelah lolos validasi, kepala jurnal (tanggal, keterangan, tipe) dimasukkan ke tabel `jurnal`, lalu setiap baris akun disimpan ke tabel `jurnal_baris` yang terhubung melalui `jurnal_id`. Nilai uang diubah ke `Decimal` lalu disimpan sebagai teks agar presisinya terjaga, dan seluruh proses ditutup dengan `conn.commit()` agar perubahan permanen.

```

database.py - next_kode & insert_jurnal()

1 def next_kode(conn: sqlite3.Connection) -> str:
2     """Kode transaksi berikutnya, format T## berdasarkan jumlah jurnal umum."""
3     cur = conn.cursor()
4     cur.execute("SELECT COUNT(*) FROM jurnal WHERE tipe = 'umum'")
5     n = cur.fetchone()[0] + 1
6     return f"T{n:02d}"
7
8
9 def insert_jurnal(
10     conn: sqlite3.Connection,
11     tanggal: str,
12     keterangan: str,
13     lines: list[dict],
14     tipe: str = "umum",
15     kode: str | None = None,
16 ) -> int:
17     """
18     Simpan satu transaksi. Validasi balance dulu (debit=kredit, ≠0).
19     Raise ValueError bila tidak balance → tidak ada yang tersimpan.
20     """
21     entry = {"id": kode or "?", "tanggal": tanggal, "keterangan": keterangan, "lines": lines}
22     acc.validasi_entry(entry) # raise ValueError jika tidak balance
23
24     if kode is None:
25         kode = next_kode(conn)
26     cur = conn.cursor()
27     cur.execute(
28         "INSERT INTO jurnal (kode, tanggal, keterangan, tipe) VALUES (?, ?, ?, ?)",
29         (kode, tanggal, keterangan, tipe),
30     )
31     jid = cur.lastrowid
32     if jid is None:
33         raise RuntimeError("Gagal menyimpan jurnal: tidak ada row id.")
34     for ln in lines:
35         cur.execute(
36             "INSERT INTO jurnal_baris (jurnal_id, kode_akun, akun, debit, kredit) "
37             "VALUES (?, ?, ?, ?, ?)",
38             (jid, ln["kode"], ln["akun"], str(D(ln["debit"])), str(D(ln["kredit"]))),
39         )
40     conn.commit()
41     return jid

```

Gambar 16. Fungsi penomoran kode dan penyimpanan jurnal pada database.py.

Penjelasan:

- next_kode menghitung COUNT jurnal umum lalu menambah 1, menghasilkan kode T## (contoh T21) secara otomatis.
- Format f"T{n:02d}" memastikan nomor selalu dua digit (T01, T02, ... T21).
- insert_jurnal memanggil acc.validasi_entry lebih dulu; bila debit tidak sama dengan kredit, ValueError dilempar.
- Karena validasi dilakukan sebelum menulis, transaksi tidak seimbang tidak akan pernah tersimpan ke basis data.
- Kepala transaksi disimpan ke tabel jurnal, lalu cur.lastrowid dipakai sebagai jurnal_id penghubung ke baris akun.

- Tiap baris akun disimpan ke `jurnal_baris` dengan nilai uang diubah ke `Decimal` lalu disimpan sebagai teks demi presisi.
- `conn.commit()` di akhir memastikan seluruh penyimpanan benar-benar permanen (tersimpan tetap).

Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.5.8 Membaca Jurnal Umum (`database.py`)

Fungsi pembacaan jurnal bertugas mengambil data transaksi dari basis data untuk ditampilkan maupun diolah menjadi laporan. Fungsi inti `_baca_jurnal` membaca kepala jurnal dari tabel `jurnal`, lalu untuk setiap kepala tersebut membaca baris-baris akunnya dari tabel `jurnal_baris`. Data disusun menjadi struktur bertingkat (dictionary): satu transaksi memiliki daftar baris akun di dalamnya. Saat dibaca, nilai uang yang tersimpan sebagai teks diubah kembali menjadi `Decimal` agar perhitungan tetap akurat. Fungsi `get_jurnal_umum` memanggil `_baca_jurnal` khusus untuk tipe “umum” sehingga hanya transaksi operasional (termasuk yang ditambah pengguna) yang diambil, sedangkan `get_semua_jurnal` mengambil seluruh tipe jurnal.

```

database.py - get_jurnal_umum & _baca_jurnal

1 def _baca_jurnal(conn: sqlite3.Connection, tipe: str | None = None) -> list[dict]:
2     cur = conn.cursor()
3     if tipe:
4         cur.execute(
5             "SELECT id, kode, tanggal, keterangan, tipe FROM jurnal WHERE tipe = ? "
6             "ORDER BY id",
7             (tipe,),
8         )
9     else:
10        cur.execute("SELECT id, kode, tanggal, keterangan, tipe FROM jurnal ORDER BY id")
11    entries = []
12    for jid, kode, tanggal, keterangan, jtipe in cur.fetchall():
13        bcur = conn.cursor()
14        bcur.execute(
15            "SELECT kode_akun, akun, debit, kredit FROM jurnal_baris "
16            "WHERE jurnal_id = ? ORDER BY id",
17            (jid,),
18        )
19        lines = [
20            {"kode": ka, "akun": ak, "debit": D(d), "kredit": D(k)}
21            for ka, ak, d, k in bcur.fetchall()
22        ]
23        entries.append({
24            "db_id": jid,
25            "id": kode,
26            "tanggal": tanggal,
27            "keterangan": keterangan,
28            "tipe": jtipe,
29            "lines": lines,
30        })
31    return entries
32
33
34 def get_jurnal_umum(conn: sqlite3.Connection) -> list[dict]:
35     """Semua jurnal bertipe 'umum' (termasuk yang ditambah manual user)."""
36     return _baca_jurnal(conn, tipe="umum")

```

Gambar 17. Fungsi pembacaan jurnal umum pada database.py.

Penjelasan:

- `_baca_jurnal` membaca kepala jurnal dahulu, lalu baris akun tiap transaksi secara berurutan (ORDER BY id).
- Parameter `tipe` (umum/penyesuaian/penutup) menyaring jurnal yang dibaca sesuai kebutuhan halaman.
- Nilai debit dan kredit yang tersimpan TEXT diubah menjadi Decimal saat dibaca agar tidak ada galat pembulatan.
- Setiap transaksi dikembalikan sebagai dictionary berisi `db_id`, `kode`, `tanggal`, `keterangan`, `tipe`, dan daftar `lines`.
- `get_jurnal_umum` memanggil `_baca_jurnal` dengan `tipe="umum"` untuk transaksi operasional sehari-hari.
- `get_semua_jurnal` memanggil tanpa filter tipe sehingga mengambil semua jenis jurnal sekaligus.

- Struktur bertingkat ini memudahkan accounting.py mengolah data menjadi buku besar dan laporan turunannya.

Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.5.9 Mengubah Jurnal (database.py)

Fungsi `update_jurnal` memungkinkan pengguna memperbaiki transaksi yang sudah tersimpan tanpa harus menghapus dan membuatnya ulang. Sama seperti saat menyimpan transaksi baru, fungsi ini terlebih dahulu memanggil `acc.validasi_entry` untuk memastikan debit dan kredit tetap seimbang; bila tidak, `ValueError` dilempar dan data lama tetap utuh. Setelah lolos validasi, fungsi memperbarui kepala jurnal (tanggal dan keterangan) pada tabel jurnal. Untuk rinciannya, pendekatan yang dipakai adalah menghapus seluruh baris akun lama lalu memasukkan baris akun yang baru, sehingga rincian selalu konsisten dengan masukan terbaru. Kode dan tipe jurnal sengaja tidak diubah agar identitas transaksi tetap terjaga, dan seluruh perubahan dikunci permanen dengan `conn.commit()`.

```

database.py - update_jurnal()

1 def update_jurnal(
2     conn: sqlite3.Connection,
3     db_id: int,
4     tanggal: str,
5     keterangan: str,
6     lines: list[dict],
7     kode: str | None = None,
8 ) -> None:
9     """
10    Perbarui satu transaksi yang sudah ada (db_id). Validasi balance dulu
11    (debit=kredit, ≠0). Raise ValueError bila tidak balance → tidak ada
12    yang berubah. Kode & tipe entry tidak diubah; hanya tanggal, keterangan,
13    dan baris-baris jurnal yang diganti.
14    """
15    entry = {"id": kode or "?", "tanggal": tanggal, "keterangan": keterangan, "lines": lines}
16    acc.validasi_entry(entry) # raise ValueError jika tidak balance → DB utuh
17
18    cur = conn.cursor()
19    cur.execute(
20        "UPDATE jurnal SET tanggal = ?, keterangan = ? WHERE id = ?",
21        (tanggal, keterangan, db_id),
22    )
23    cur.execute("DELETE FROM jurnal_baris WHERE jurnal_id = ?", (db_id,))
24    for ln in lines:
25        cur.execute(
26            "INSERT INTO jurnal_baris (jurnal_id, kode_akun, akun, debit, kredit) "
27            "VALUES (?, ?, ?, ?, ?)",
28            (db_id, ln["kode"], ln["akun"], str(D(ln["debit"])), str(D(ln["kredit"]))),
29        )
30    conn.commit()

```

Gambar 18. Fungsi pembaruan jurnal pada database.py.

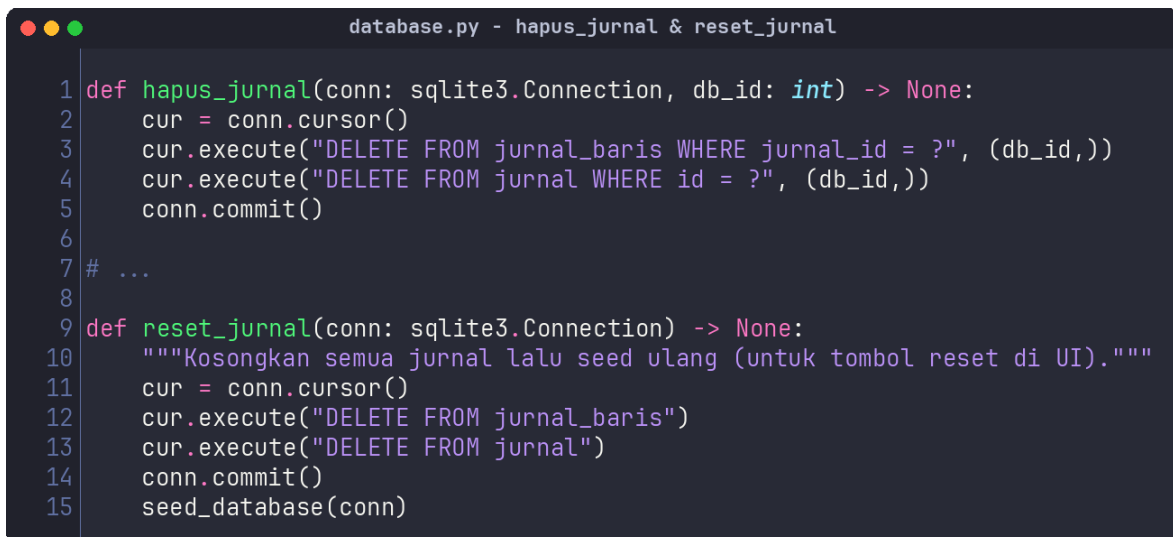
Penjelasan:

- `acc.validasi_entry` dipanggil di awal sehingga pembaruan tidak seimbang ditolak dan data lama tetap aman.
- Perintah `UPDATE jurnal SET tanggal, keterangan` memperbarui kepala transaksi sesuai masukan baru.
- Seluruh baris akun lama dihapus dengan `DELETE FROM jurnal_baris WHERE jurnal_id = ?`.
- Baris akun baru kemudian dimasukkan ulang, menjamin rincian selalu cocok dengan data terbaru.
- Kode dan tipe jurnal tidak diubah agar identitas transaksi (misalnya T21) tetap konsisten.
- Nilai uang tetap diubah ke `Decimal` lalu disimpan sebagai teks, sama seperti saat penyimpanan awal.
- `conn.commit()` memastikan pembaruan kepala dan rincian tersimpan sebagai satu kesatuan yang utuh.

Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.5.10 Menghapus dan Mereset Jurnal (database.py)

Bagian ini memuat dua fungsi pengelolaan data, yaitu `hapus_jurnal` untuk menghapus satu transaksi dan `reset_jurnal` untuk mengembalikan seluruh data ke kondisi awal. Fungsi `hapus_jurnal` mula-mula menghapus seluruh baris akun milik transaksi tersebut dari tabel `jurnal_baris`, baru kemudian menghapus kepala transaksinya dari tabel `jurnal`, sehingga tidak ada baris akun yatim (baris yang induknya sudah tidak ada). Fungsi `reset_jurnal` mengosongkan kedua tabel `jurnal` lalu memanggil `seed_database` untuk mengisi kembali 20 transaksi contoh. Reset ini berguna ketika pengguna ingin berlatih dari awal atau membersihkan transaksi percobaan. Karena bersifat menghapus banyak data, fitur reset di antarmuka diberi konfirmasi terlebih dahulu agar tidak terpicu tanpa sengaja.



```

database.py - hapus_jurnal & reset_jurnal

1 def hapus_jurnal(conn: sqlite3.Connection, db_id: int) -> None:
2     cur = conn.cursor()
3     cur.execute("DELETE FROM jurnal_baris WHERE jurnal_id = ?", (db_id,))
4     cur.execute("DELETE FROM jurnal WHERE id = ?", (db_id,))
5     conn.commit()
6
7 # ...
8
9 def reset_jurnal(conn: sqlite3.Connection) -> None:
10     """Kosongkan semua jurnal lalu seed ulang (untuk tombol reset di UI)."""
11     cur = conn.cursor()
12     cur.execute("DELETE FROM jurnal_baris")
13     cur.execute("DELETE FROM jurnal")
14     conn.commit()
15     seed_database(conn)

```

Gambar 19. Fungsi penghapusan dan reset jurnal pada database.py.

Penjelasan:

- hapus_jurnal menghapus baris akun lebih dulu (jurnal_baris) baru kepala transaksinya (jurnal).
- Urutan ini mencegah munculnya baris akun yatim, yaitu rincian yang induk transaksinya sudah terhapus.
- reset_jurnal menjalankan DELETE pada jurnal_baris dan jurnal untuk mengosongkan seluruh data transaksi.
- Setelah dikosongkan, reset_jurnal memanggil seed_database untuk mengisi ulang 20 transaksi contoh (T01-T20).
- Reset bermanfaat untuk berlatih dari nol atau menghapus transaksi percobaan yang menumpuk.
- Karena menghapus banyak data sekaligus, fitur reset di antarmuka memakai kotak konfirmasi sebelum dijalankan.
- conn.commit() pada kedua fungsi memastikan penghapusan benar-benar tersimpan ke berkas basis data.

Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.5.11 Menyimpan Transaksi dari Formulir (pages_input.py)

Berkas `pages_input.py` adalah halaman tempat pengguna mencatat transaksi baru, dan ia dirancang dengan pengamanan ketat. Formulir input meminta tanggal, keterangan, satu baris akun debit beserta nominalnya, dan satu baris akun kredit beserta nominalnya. Akun tidak diketik bebas melainkan dipilih dari bagan akun resmi (chart of accounts) melalui `st.selectbox`, sehingga pengguna tidak mungkin memasukkan akun di luar 15 akun yang sah. Ketika tombol simpan ditekan, kedua baris dirakit menjadi daftar lines lalu dikirim ke `db.insert_jurnal` yang memvalidasi keseimbangan debit-kredit. Bila tidak seimbang, pesan kesalahan ditampilkan dan tidak ada yang tersimpan; bila berhasil, muncul notifikasi sukses dan halaman dimuat ulang agar daftar transaksi langsung diperbarui.


```
pages_input.py - simpan transaksi

1  with st.form("form_input_transaksi", clear_on_submit=True):
2      tanggal = st.date_input("Tanggal", value=datetime.date.today())
3      keterangan = st.text_input("Keterangan", placeholder="Contoh: Pembelian benih padi")
4
5      st.markdown("**Baris Debit**")
6      kol_d1, kol_d2 = st.columns([2, 1])
7      with kol_d1:
8          akun_debit = st.selectbox(
9              "Akun Debit", chart, format_func=_label, key="akun_debit"
10             )
11      with kol_d2:
12          nominal_debit = st.number_input(
13              "Nominal Debit", min_value=0, step=1000, key="nominal_debit"
14          )
15
16      st.markdown("**Baris Kredit**")
17      kol_k1, kol_k2 = st.columns([2, 1])
18      with kol_k1:
19          akun_kredit = st.selectbox(
20              "Akun Kredit", chart, format_func=_label, key="akun_kredit"
21          )
22      with kol_k2:
23          nominal_kredit = st.number_input(
24              "Nominal Kredit", min_value=0, step=1000, key="nominal_kredit"
25          )
26
27      simpan = st.form_submit_button("Simpan Transaksi")
28
29  if simpan:
30      lines = [
31          {
32              "kode": akun_debit["kode"],
33              "akun": akun_debit["akun"],
34              "debit": int(nominal_debit),
35              "kredit": 0,
36          },
37          {
38              "kode": akun_kredit["kode"],
39              "akun": akun_kredit["akun"],
40              "debit": 0,
41              "kredit": int(nominal_kredit),
42          },
43      ]
44      try:
45          db.insert_jurnal(
46              conn,
47              tanggal.isoformat(),
48              keterangan,
49              lines,
50              tipe="umum",
51          )
52      except ValueError as e:
53          st.error(str(e))
54      else:
55          st.success("Transaksi berhasil disimpan.")
56          st.rerun()
```

Gambar 20. Proses penyimpanan transaksi pada pages_input.py.

Penjelasan:

- Formulir dibungkus st.form dengan clear_on_submit=True agar kolom otomatis kosong setelah transaksi tersimpan.
- Akun debit dan kredit dipilih dari bagan akun lewat st.selectbox, bukan diketik bebas, sehingga selalu sah.

- Nominal dimasukkan lewat `st.number_input` dengan langkah (step) 1000 dan nilai minimal 0.
- Dua baris (debit dan kredit) dirakit menjadi daftar lines berisi kode, nama akun, debit, dan kredit.
- `db.insert_jurnal` dipanggil di dalam blok try; `ValueError` dari validasi ditangkap dan ditampilkan sebagai `st.error`.
- Bila tidak seimbang, tidak ada data tersimpan; bila berhasil, `st.success` muncul dan `st.rerun` memuat ulang daftar.
- Pembatasan akun ke bagan resmi menjaga konsistensi laporan karena setiap akun sudah punya tipe dan saldo normal yang jelas.

Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.5.12 Tombol Ubah dan Hapus (`pages_input.py`)

Pada daftar transaksi, setiap baris dilengkapi tombol untuk mengubah atau menghapus agar pengelolaan data terasa mudah. Untuk menjaga keutuhan data contoh, transaksi seed (T01-T20) dikunci dan hanya diberi label “data awal” tanpa tombol, sedangkan transaksi tambahan buatan pengguna memiliki tombol Edit dan Hapus. Ketika tombol Edit ditekan, `db_id` transaksi terpilih disimpan ke `session_state.edit_db_id` sehingga aplikasi tahu transaksi mana yang sedang disunting setelah halaman dimuat ulang. Ketika tombol Hapus ditekan, fungsi `db.hapus_jurnal` langsung dipanggil untuk menghapus transaksi tersebut. Pemisahan antara data terkunci dan data yang bisa diubah ini melindungi contoh bawaan agar tidak terhapus tanpa sengaja.

```
pages_input.py - tombol edit & hapus

1  for e in entries:
2      is_seed = e["id"] in seed_codes
3      kol_info, kol_edit, kol_hapus = st.columns([5, 1, 1])
4      with kol_info:
5          st.markdown(
6              f"*{e['id']}* · {e['tanggal']} · {e['keterangan']} "
7              f"· {acc.format_rupiah(_total_debit(e))}"
8          )
9      if is_seed:
10         with kol_edit:
11             st.caption("data awal")
12     else:
13         with kol_edit:
14             if st.button("Edit", key=f"edit_{e['db_id']}"):
15                 st.session_state.edit_db_id = e["db_id"]
16                 st.rerun()
17         with kol_hapus:
18             if st.button("Hapus", key=f"hapus_{e['db_id']}"):
19                 db.hapus_jurnal(conn, e["db_id"])
20                 if st.session_state.edit_db_id == e["db_id"]:
21                     st.session_state.edit_db_id = None
22                 st.rerun()
```

Gambar 21. Tombol ubah dan hapus transaksi pada `pages_input.py`.

Penjelasan:

- Himpunan kode seed (T01-T20) dipakai untuk membedakan transaksi contoh dari transaksi tambahan pengguna.
- Transaksi seed ditandai sebagai “data awal” (terkunci) dan tidak diberi tombol agar tidak bisa diubah atau dihapus.
- Tiap baris ditata dalam tiga kolom (informasi, tombol edit, tombol hapus) memakai `st.columns`.
- Tombol Edit menyimpan `db_id` transaksi ke `st.session_state.edit_db_id` lalu memanggil `st.rerun`.
- `session_state` (memori sesi) menjaga transaksi mana yang sedang disunting meski halaman dimuat ulang.
- Tombol Hapus memanggil `db.hapus_jurnal(conn, db_id)` untuk menghapus transaksi beserta seluruh barisnya.
- Setiap tombol diberi key unik (mis. `edit_<db_id>`) agar Streamlit tidak mencampuradukkan tombol antar-transaksi.

Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.5.13 Formulir Ubah Transaksi (pages_input.py)

Ketika sebuah transaksi tambahan dipilih untuk diubah, fungsi `_render_form_edit` menampilkan formulir yang sudah terisi (pre-fill) dengan nilai transaksi saat ini. Fungsi ini cerdas mengenali baris mana yang debit dan mana yang kredit, lalu menempatkan akun yang benar pada masing-masing selectbox sebagai pilihan awal. Tanggal lama juga dimuat kembali, dengan pengaman bila format tanggalnya tidak valid maka dipakai tanggal hari ini. Pengguna dapat menyunting tanggal, keterangan, akun, dan nominal, lalu menekan Simpan Perubahan atau Batal. Saat disimpan, data dikirim ke `db.update_jurnal` yang kembali memvalidasi keseimbangan; bila gagal ditampilkan pesan error, bila berhasil mode edit ditutup dan halaman dimuat ulang. Tombol Batal sekadar mengosongkan status edit tanpa menyimpan apa pun.

```
pages_input.py - form edit transaksi
1  with st.container(border=True):
2      st.markdown(f"### Edit Transaksi `{entry['id']}`")
3      st.caption("Akun dipilih dari bagan akun resmi. Debit harus sama dengan kredit.")
4      with st.form("form_edit_transaksi"):
5          tanggal = st.date_input("Tanggal", value=tgl_awal, key=f"edit_tanggal_{did}")
6          keterangan = st.text_input(
7              "Keterangan", value=entry["keterangan"], key=f"edit_keterangan_{did}"
8          )
9
10         st.markdown("**Baris Debit**")
11         kol_d1, kol_d2 = st.columns([2, 1])
12         with kol_d1:
13             akun_debit = st.selectbox(
14                 "Akun Debit", chart, format_func=label_fn,
15                 index=idx_debit, key=f"edit_akun_debit_{did}",
16             )
17         with kol_d2:
18             nominal_debit = st.number_input(
19                 "Nominal Debit", min_value=0, step=1000,
20                 value=int(debit_line["debit"]), key=f"edit_nominal_debit_{did}",
21             )
22
23         st.markdown("**Baris Kredit**")
24         kol_k1, kol_k2 = st.columns([2, 1])
25         with kol_k1:
26             akun_kredit = st.selectbox(
27                 "Akun Kredit", chart, format_func=label_fn,
28                 index=idx_kredit, key=f"edit_akun_kredit_{did}",
29             )
30         with kol_k2:
31             nominal_kredit = st.number_input(
32                 "Nominal Kredit", min_value=0, step=1000,
33                 value=int(kredit_line["kredit"]), key=f"edit_nominal_kredit_{did}",
34             )
35
36         kol_simpan, kol_batal = st.columns(2)
37         with kol_simpan:
38             simpan = st.form_submit_button("Simpan Perubahan")
39         with kol_batal:
40             batal = st.form_submit_button("Batal")
41
42     if batal:
43         st.session_state.edit_db_id = None
44         st.rerun()
```

Gambar 22. Formulir ubah transaksi pada `pages_input.py`.

Penjelasan:

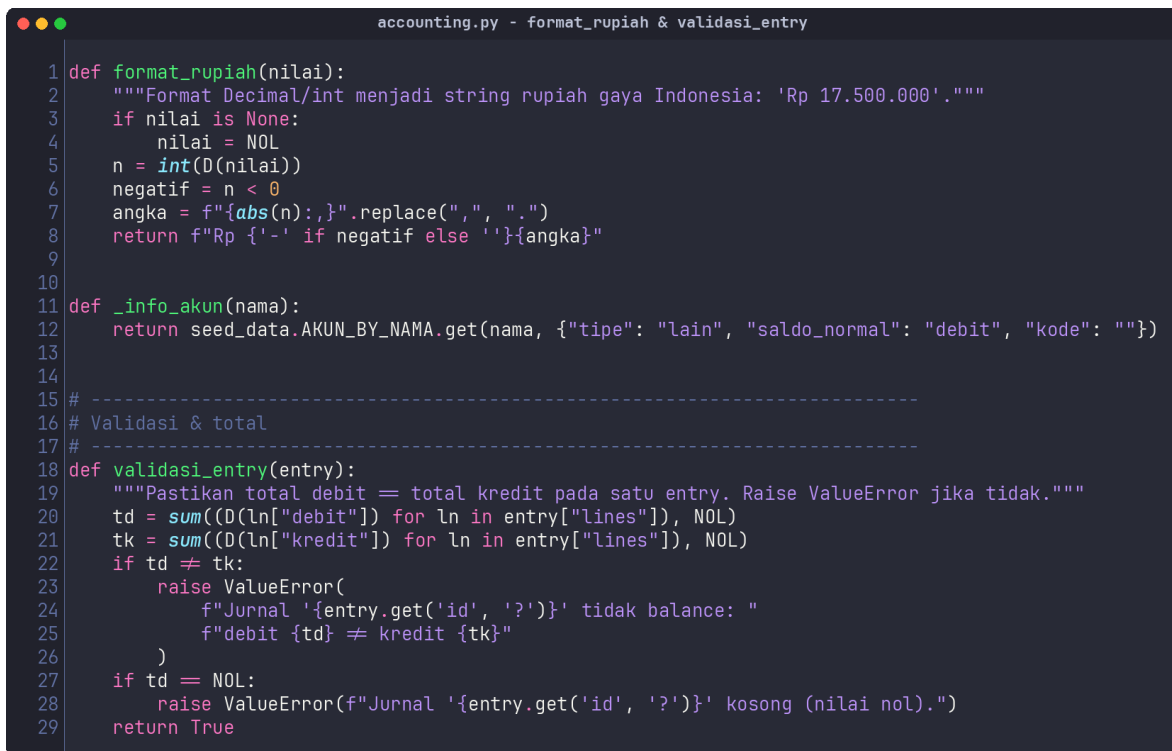
- Baris debit dan kredit dikenali otomatis dengan mencari baris yang nilai debit atau kreditnya lebih dari nol.
- Akun yang sedang dipakai ditempatkan sebagai pilihan awal selectbox lewat perhitungan index yang cocok.
- Tanggal lama dimuat dengan `datetime.date.fromisoformat`; bila formatnya rusak dipakai tanggal hari ini sebagai cadangan.
- Setiap kolom input diberi key yang mengandung `db_id` agar formulir antar transaksi tidak tertukar.

- Tombol “Simpan Perubahan” merakit ulang baris lalu memanggil `db.update_jurnal` di dalam blok `try`.
- Jika validasi gagal, `st.error` menampilkan pesan; jika berhasil, `edit_db_id` dikosongkan dan `st.rerun` dipanggil.
- Tombol “Batal” hanya menyetel `edit_db_id` ke `None` sehingga formulir tertutup tanpa mengubah data.

Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.5.14 Format Rupiah dan Validasi Entry (`accounting.py`)

Berkas `accounting.py` adalah mesin akuntansi inti aplikasi, dan bagian ini memuat dua fungsi pendukung yang sering dipakai. Fungsi `format_rupiah` mengubah angka (Decimal atau int) menjadi teks mata uang gaya Indonesia, misalnya nilai 17500000 menjadi “Rp 17.500.000”, lengkap dengan tanda titik pemisah ribuan dan dukungan nilai negatif. Fungsi `validasi_entry` adalah penjaga integritas transaksi: ia menjumlahkan seluruh debit dan seluruh kredit pada satu transaksi, lalu memastikan keduanya sama dan tidak nol. Bila debit tidak sama dengan kredit, atau total bernilai nol, fungsi melempar `ValueError` dengan pesan yang menjelaskan masalahnya. Karena validasi ini dipanggil sebelum setiap penyimpanan dan pembaruan, prinsip dasar akuntansi berpasangan ($\text{debit} = \text{kredit}$) selalu terjaga.



```

accounting.py - format_rupiah & validasi_entry

1 def format_rupiah(nilai):
2     """Format Decimal/int menjadi string rupiah gaya Indonesia: 'Rp 17.500.000'."""
3     if nilai is None:
4         nilai = NOL
5     n = int(D(nilai))
6     negatif = n < 0
7     angka = f"{abs(n):,}".replace(",", ".")
8     return f"Rp {'-' if negatif else ''}{angka}"
9
10
11 def _info_akun(nama):
12     return seed_data.AKUN_BY_NAMA.get(nama, {"tipe": "lain", "saldo_normal": "debit", "kode": ""})
13
14
15 # -----
16 # Validasi & total
17 # -----
18 def validasi_entry(entry):
19     """Pastikan total debit = total kredit pada satu entry. Raise ValueError jika tidak."""
20     td = sum((D(ln["debit"]) for ln in entry["lines"]), NOL)
21     tk = sum((D(ln["kredit"]) for ln in entry["lines"]), NOL)
22     if td != tk:
23         raise ValueError(
24             f"Jurnal '{entry.get('id', '?')}' tidak balance: "
25             f"debit {td} != kredit {tk}"
26         )
27     if td == NOL:
28         raise ValueError(f"Jurnal '{entry.get('id', '?')}' kosong (nilai nol).")
29     return True

```

Gambar 23. Fungsi format rupiah dan validasi entry pada accounting.py.

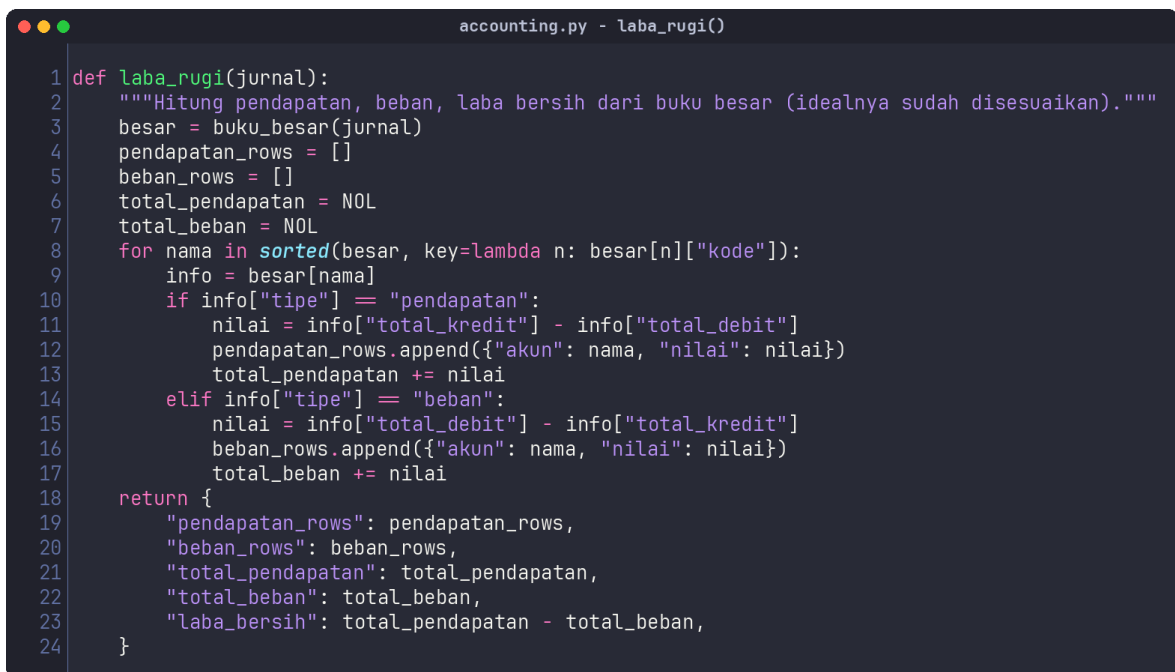
Penjelasan:

- format_rupiah mengubah Decimal/int menjadi teks seperti “Rp 17.500.000” dengan titik sebagai pemisah ribuan.
- Tanda titik diperoleh dengan mengganti koma pemformatan (f"{abs(n):,}") menjadi titik agar sesuai gaya Indonesia.
- format_rupiah juga menangani nilai negatif dengan menambahkan tanda minus di depan angka.
- validasi_entry menjumlahkan seluruh debit dan seluruh kredit memakai tipe Decimal agar presisi.
- Bila total debit tidak sama dengan total kredit, ValueError dilempar beserta pesan selisihnya.
- Bila total bernilai nol (transaksi kosong), ValueError juga dilempar agar tidak ada jurnal hampa tersimpan.
- Karena dipanggil sebelum setiap simpan dan ubah, fungsi ini menjamin prinsip akuntansi berpasangan (debit = kredit) selalu terpenuhi.

Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.5.15 Penyusunan Laporan Laba Rugi (accounting.py)

Fungsi laba_rugi menghasilkan salah satu laporan terpenting bagi petani, yaitu seberapa besar untung atau rugi usaha tani pada satu periode. Fungsi ini pertama-tama membangun buku besar dari daftar jurnal agar saldo tiap akun diketahui. Kemudian ia menelusuri seluruh akun: untuk akun bertipe pendapatan, nilainya dihitung dari selisih kredit dikurangi debit; untuk akun bertipe beban, nilainya dihitung dari selisih debit dikurangi kredit. Seluruh pendapatan dan seluruh beban dijumlahkan masing-masing, lalu laba bersih diperoleh dari total pendapatan dikurangi total beban. Hasilnya dikembalikan sebagai dictionary berisi rincian baris pendapatan, baris beban, totalnya, serta laba bersih, sehingga halaman laporan tinggal menampilkannya dengan rapi.



```
accounting.py - laba_rugi()

1 def laba_rugi(jurnal):
2     """Hitung pendapatan, beban, laba bersih dari buku besar (idealnya sudah disesuaikan)."""
3     besar = buku_besar(jurnal)
4     pendapatan_rows = []
5     beban_rows = []
6     total_pendapatan = NOL
7     total_beban = NOL
8     for nama in sorted(besar, key=lambda n: besar[n]["kode"]):
9         info = besar[nama]
10        if info["tipe"] == "pendapatan":
11            nilai = info["total_kredit"] - info["total_debit"]
12            pendapatan_rows.append({"akun": nama, "nilai": nilai})
13            total_pendapatan += nilai
14        elif info["tipe"] == "beban":
15            nilai = info["total_debit"] - info["total_kredit"]
16            beban_rows.append({"akun": nama, "nilai": nilai})
17            total_beban += nilai
18    return {
19        "pendapatan_rows": pendapatan_rows,
20        "beban_rows": beban_rows,
21        "total_pendapatan": total_pendapatan,
22        "total_beban": total_beban,
23        "laba_bersih": total_pendapatan - total_beban,
24    }
```

Gambar 24. Fungsi penyusunan laporan laba rugi pada accounting.py.

Penjelasan:

- Fungsi dimulai dengan membangun buku_besar(jurnal) agar saldo setiap akun tersedia lebih dulu.

- Akun diurutkan berdasarkan kode akun supaya urutan baris laporan konsisten dan mudah dibaca.
- Untuk akun pendapatan, nilai dihitung sebagai `total_kredit` dikurangi `total_debit` (saldo normal kredit).
- Untuk akun beban, nilai dihitung sebagai `total_debit` dikurangi `total_kredit` (saldo normal debit).
- `total_pendapatan` dan `total_beban` dijumlahkan terpisah memakai tipe `Decimal` agar tetap presisi.
- `laba_bersih` dihitung sebagai `total_pendapatan` dikurangi `total_beban`, menunjukkan untung (positif) atau rugi (negatif).
- Hasil dikemas dalam dictionary (baris pendapatan, baris beban, total, dan laba bersih) agar mudah ditampilkan halaman laporan.

Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.5.16 Pengambilan Data Laporan (`ui_helpers.py`)

Fungsi `get_data` pada `ui_helpers.py` berperan sebagai satu sumber kebenaran (single source of truth) untuk seluruh data laporan, sehingga semua halaman memakai data yang sama dan konsisten. Pertama, ia membaca jurnal umum dari basis data melalui `db.get_jurnal_umum`. Lalu ia menyusun versi “disesuaikan” dengan menggabungkan jurnal umum dan jurnal penyesuaian. Dari versi disesuaikan itu, ia menghitung jurnal penutup memakai `acc.jurnal_penutup`, dan terakhir menyusun versi “setelah penutupan” dengan menggabungkan jurnal disesuaikan dan jurnal penutup. Keempat himpunan data ini dikembalikan dalam satu dictionary, sehingga setiap halaman laporan tinggal mengambil bagian yang dibutuhkannya tanpa menghitung ulang dari awal.

```

1 def get_data(conn):
2     """
3     Bangun seluruh alur data jurnal dari koneksi DB.
4
5     Return dict dengan 4 key:
6     jurnal          -> jurnal umum dari DB (20 entri pada data seed)
7     disesuaikan     -> jurnal umum + jurnal penyesuaian (21 entri)
8     penutup         -> jurnal penutup dari jurnal disesuaikan (4 entri)
9     setelah_penutupan -> disesuaikan + penutup
10    """
11    jurnal = db.get_jurnal_umum(conn)
12    disesuaikan = jurnal + acc.get_jurnal_penyesuaian()
13    penutup = acc.jurnal_penutup(disesuaikan)
14    setelah_penutupan = disesuaikan + penutup
15    return {
16        "jurnal": jurnal,
17        "disesuaikan": disesuaikan,
18        "penutup": penutup,
19        "setelah_penutupan": setelah_penutupan,
20    }

```

Gambar 25. Fungsi `get_data` untuk menyiapkan data laporan pada `ui_helpers.py`.

Penjelasan:

- `get_data` membaca jurnal umum dari basis data sebagai titik awal seluruh perhitungan laporan.
- Versi “disesuaikan” dibentuk dengan menambahkan jurnal penyesuaian ke jurnal umum.
- Jurnal penutup dihitung dari versi disesuaikan memakai `acc.jurnal_penutup`.
- Versi “setelah penutupan” dibentuk dari jurnal disesuaikan ditambah jurnal penutup.
- Keempat himpunan (`jurnal`, `disesuaikan`, `penutup`, `setelah_penutupan`) dikembalikan dalam satu dictionary.
- Modul ini bersifat read-only: ia hanya membaca dan merangkai data, tidak pernah menulis ke basis data.
- Dengan satu sumber data terpusat, seluruh halaman laporan dijamin menampilkan angka yang konsisten satu sama lain.

Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.5.17 Penyajian Tabel Berstyle (ui_helpers.py)

Agar setiap laporan tampil rapi dan seragam, `ui_helpers.py` menyediakan fungsi `tabel_html` yang mengubah data menjadi tabel HTML berstyle. Fungsi ini menerima data baik dalam bentuk `DataFrame` pandas maupun `list-of-dict`, lalu membangun bagian kepala (`thead`) dan badan (`tbody`) tabel. Kolom angka dapat ditandai sebagai `right_cols` sehingga dirata-kanan dan memakai angka tabular agar kolom rupiah mudah dibandingkan secara vertikal. Demi keamanan, setiap isi sel dilewatkan `html.escape()` untuk mencegah serangan XSS (penyisipan kode berbahaya), terutama pada kolom Keterangan yang berisi teks bebas dari pengguna. Fungsi juga dapat menandai baris total agar dicetak tebal, sehingga ringkasan laporan langsung terlihat menonjol dengan warna tema hijau yang konsisten.

```

ui_helpers.py - tabel_html()

1 def tabel_html(rows_or_df, right_cols=None, total_row_label=None, headers=None):
2     """Bangun string HTML tabel berstyle dari DataFrame ATAU list-of-dict.
3
4     Parameter:
5         rows_or_df      : pandas.DataFrame atau list[dict] berisi baris tabel.
6         right_cols       : iterable nama kolom yang dirata-kanan (kolom angka)
7                           → mendapat class .ajp-num (tabular-nums, nowrap).
8         total_row_label  : bila diset, baris yang sel KOLOM PERTAMA-nya sama dengan
9                           nilai ini ditandai <tr class="ajp-total"> (tebal).
10        headers          : override urutan/nama kolom (opsional).
11
12    KEAMANAN: setiap sel header DAN body dilewatkan html.escape() – kolom
13    "Keterangan" memuat teks bebas input manual pengguna (permukaan XSS).
14    Nilai yang sudah diformat acc.format_rupiah dipakai apa adanya
15    (tidak di-parse / di-format ulang).
16    """
17    right_cols = set(right_cols or [])
18
19    if isinstance(rows_or_df, pd.DataFrame):
20        cols = list(headers) if headers is not None else list(rows_or_df.columns)
21        records = rows_or_df.to_dict("records")
22    else:
23        records = list(rows_or_df or [])
24        if headers is not None:
25            cols = list(headers)
26        elif records:
27            cols = list(records[0].keys())
28        else:
29            cols = []
30
31    # thead – header bertanda .ajp-num bila kolom angka (rata kanan konsisten)
32    head_cells = []
33    for c in cols:
34        cls = ' class="ajp-num"' if c in right_cols else ""
35        head_cells.append(f"<th{cls}>{html.escape(_bersihkan_sel(c))}</th>")
36    thead = "<thead><tr>" + "".join(head_cells) + "</tr></thead>"
37
38    # tbody
39    body_rows = []
40    first_col = cols[0] if cols else None
41    for rec in records:
42        is_total = False
43        if total_row_label is not None and first_col is not None:
44            label_val = _bersihkan_sel(rec.get(first_col, "")) if isinstance(rec, dict) else ""
45            if label_val == str(total_row_label):
46                is_total = True
47        tr_cls = ' class="ajp-total"' if is_total else ""
48        cells = []
49        for c in cols:
50            val = _bersihkan_sel(rec.get(c, "")) if isinstance(rec, dict) else ""
51            cls = ' class="ajp-num"' if c in right_cols else ""
52            cells.append(f"<td{cls}>{html.escape(val)}</td>")
53        body_rows.append(f"<tr{tr_cls}>" + "".join(cells) + "</tr>")
54    tbody = "<tbody>" + "".join(body_rows) + "</tbody>"
55
56    return (
57        '<div class="ajp-table-wrap"><table class="ajp-table">'
58        + thead
59        + tbody
60        + "</table></div>"
61    )

```

Gambar 26. Fungsi penyajian tabel HTML berstyle pada ui_helpers.py.

Penjelasan:

- tabel_html menerima data berupa DataFrame pandas maupun list-of-dict, lalu menyusunnya jadi tabel HTML.

- Parameter `right_cols` menandai kolom angka agar dirata-kanan dengan class `.ajp-num` (angka tabular, tidak terpotong).
- Perataan kanan membuat kolom rupiah mudah dibandingkan dari atas ke bawah.
- Setiap sel header dan isi dilewatkan `html.escape()` untuk mencegah XSS (penyisipan kode berbahaya).
- Kolom Keterangan memuat teks bebas dari pengguna, sehingga escape ini menjadi pengaman penting.
- Parameter `total_row_label` menandai baris total dengan class `.ajp-total` sehingga dicetak tebal dan menonjol.
- Tabel dibungkus div `.ajp-table-wrap` sehingga mewarisi gaya tema hijau yang seragam di seluruh laporan.

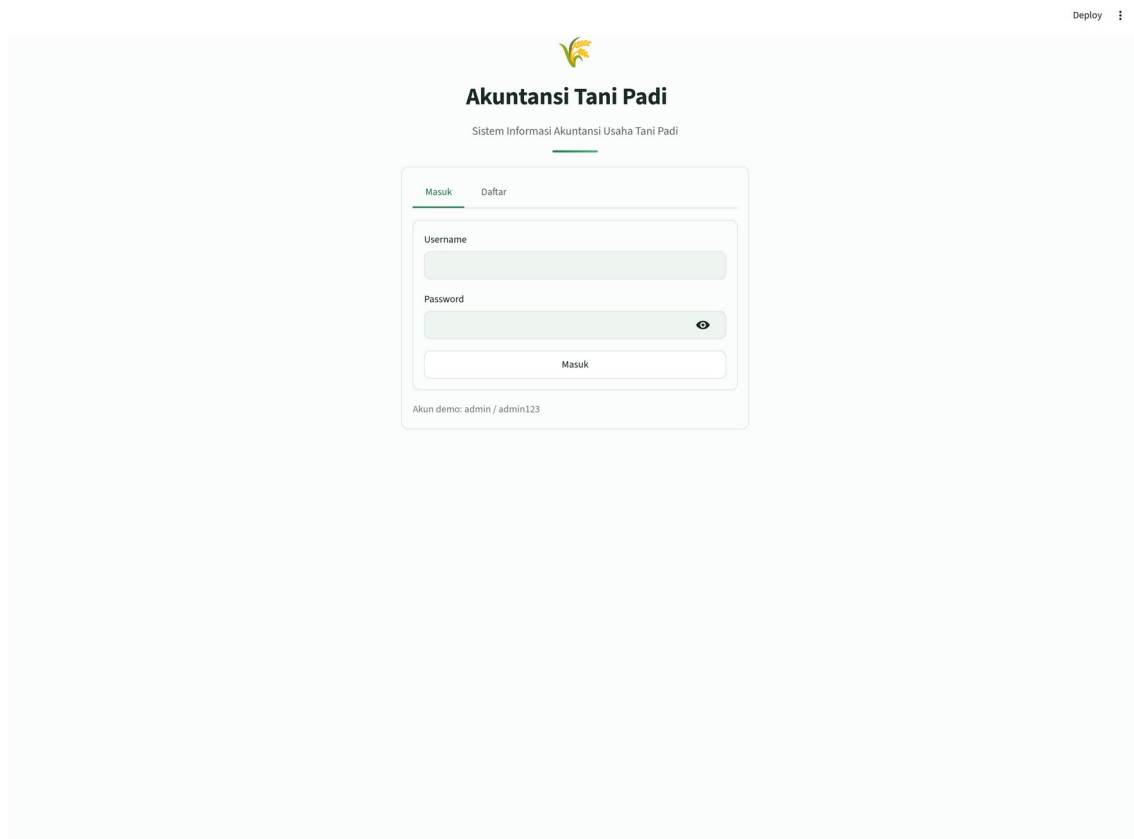
Singkatnya, bagian kode ini menjadi salah satu fondasi penting agar aplikasi Akuntansi Tani Padi bekerja dengan benar, aman, dan mudah digunakan oleh pelaku usaha tani.

2.6 Panduan Penggunaan Aplikasi

Bagian ini menjelaskan langkah penggunaan aplikasi disertai tangkapan layar dari aplikasi yang sedang berjalan. Pengguna dapat mengikuti urutan berikut untuk memanfaatkan seluruh fitur aplikasi.

2.6.1 Halaman Login

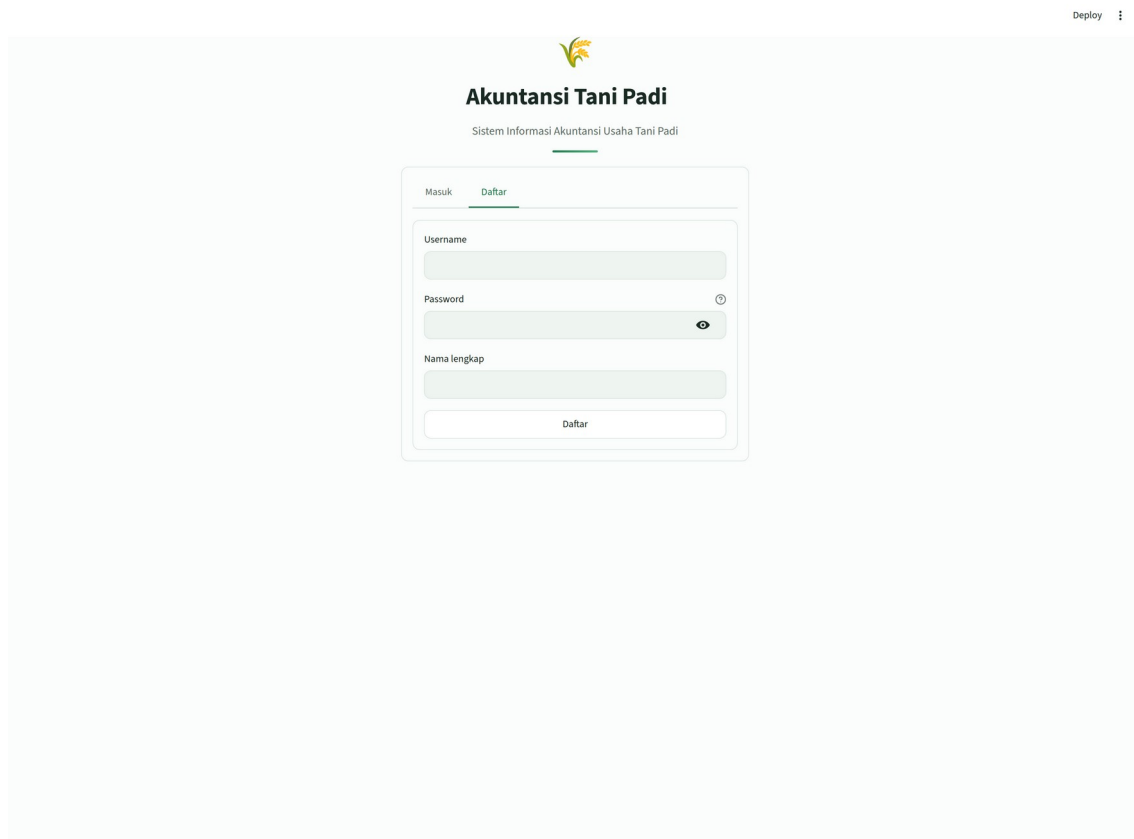
Saat pertama kali dibuka, aplikasi menampilkan halaman login. Pengguna memasukkan username dan kata sandi, lalu menekan tombol Login untuk masuk ke aplikasi.



Gambar 27. Halaman login aplikasi.

2.6.2 Halaman Pendaftaran

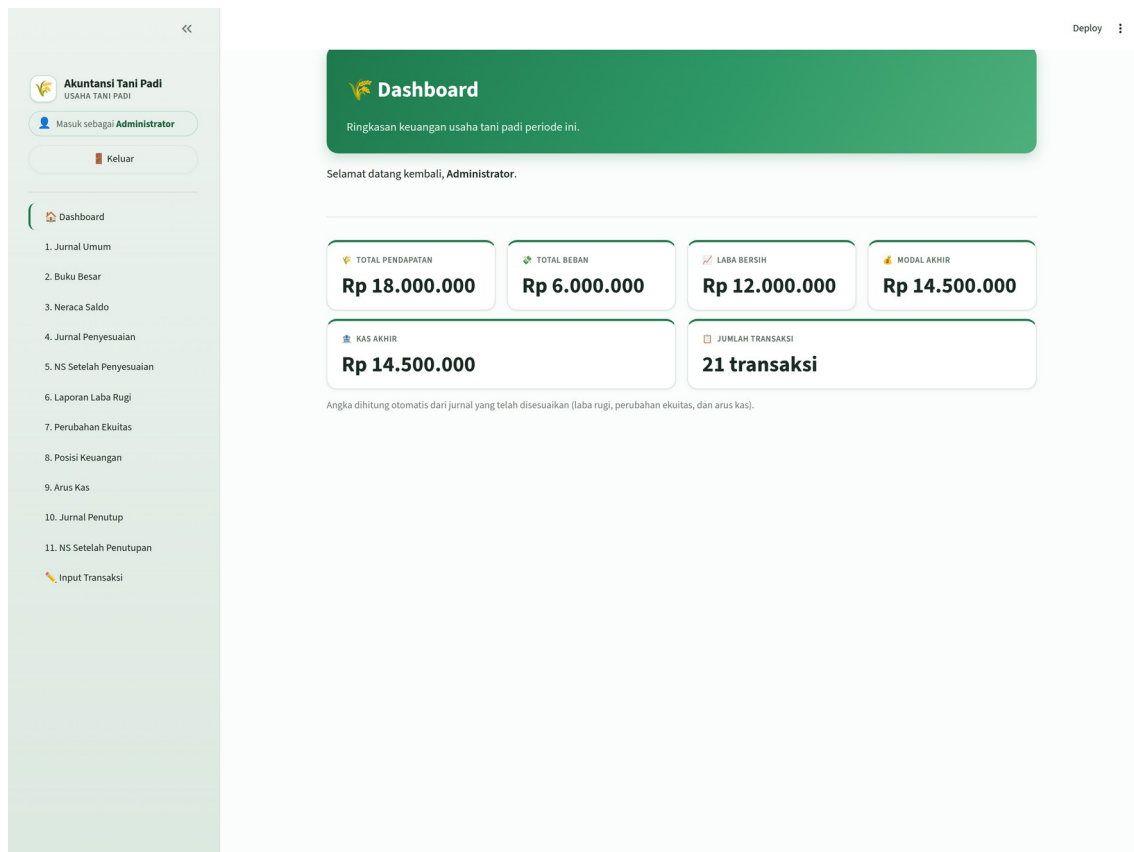
Pengguna baru dapat membuat akun melalui halaman pendaftaran dengan mengisi nama, username, dan kata sandi.



Gambar 28. Halaman pendaftaran pengguna baru.

2.6.3 Dasbor Ringkasan

Setelah berhasil login, pengguna disambut oleh dasbor yang menampilkan ringkasan keuangan, meliputi total pendapatan, total beban, laba bersih, dan saldo kas.



Gambar 29. Dasbor ringkasan keuangan.

2.6.4 Jurnal Umum

Menu Jurnal Umum menampilkan seluruh transaksi yang telah dicatat beserta akun debit dan kreditnya secara berurutan.

<<

Akuntansi Tani Padi

USANA TANI PADI

Masuk sebagai Administrator

Keluar

Dashboard

1. Jurnal Umum

2. Buku Besar

3. Neraca Saldo

4. Jurnal Penyesuaian

5. NS Setelah Penyesuaian

6. Laporan Laba Rugi

7. Perubahan Ekuitas

8. Posisi Keuangan

9. Arus Kas

10. Jurnal Penutup

11. NS Setelah Penutupan

Input Transaksi

Jurnal Umum

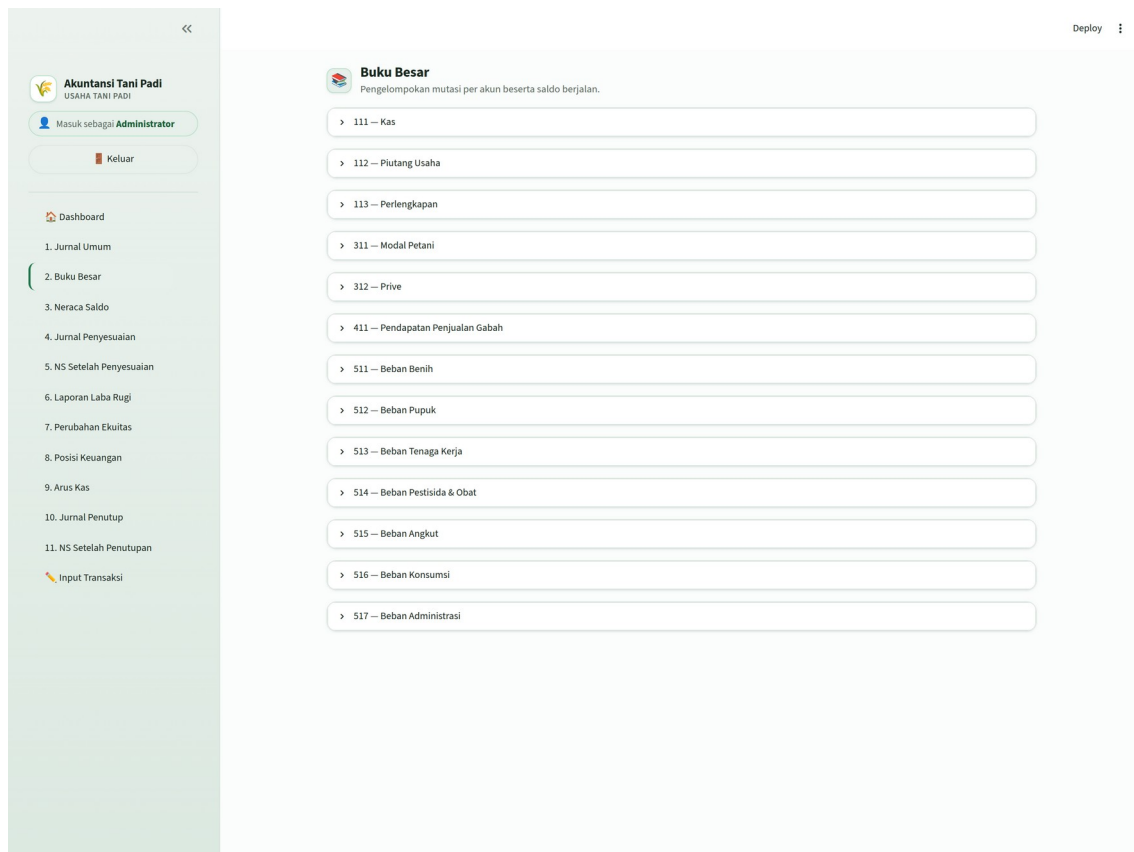
Pencatatan kronologis seluruh transaksi sebelum penyesuaian (21 transaksi).

Tanggal	Kode	Keterangan	Akun	Debit	Kredit
2025-01-05	T01	Setoran modal awal pemilik ke kas usaha tani	Kas	Rp 7.000.000	
2025-01-05	T01	Setoran modal awal pemilik ke kas usaha tani	Modal Petani		Rp 7.000.000
2025-01-07	T02	Pembelian benih padi Mekongga 10 kantong (50 kg)	Beban Benih	Rp 500.000	
2025-01-07	T02	Pembelian benih padi Mekongga 10 kantong (50 kg)	Kas		Rp 500.000
2025-01-10	T03	Upah tenaga kerja pembajakan & persiapan lahan 1,5 ha	Beban Tenaga Kerja	Rp 1.000.000	
2025-01-10	T03	Upah tenaga kerja pembajakan & persiapan lahan 1,5 ha	Kas		Rp 1.000.000
2025-01-12	T04	Pembelian pupuk Urea Pusri gelombang pertama	Beban Pupuk	Rp 400.000	
2025-01-12	T04	Pembelian pupuk Urea Pusri gelombang pertama	Kas		Rp 400.000
2025-01-13	T05	Pembelian pupuk TSP untuk dasar tanaman	Beban Pupuk	Rp 300.000	
2025-01-13	T05	Pembelian pupuk TSP untuk dasar tanaman	Kas		Rp 300.000
2025-01-14	T06	Pembelian pupuk ZA sebagai campuran tambahan	Beban Pupuk	Rp 200.000	
2025-01-14	T06	Pembelian pupuk ZA sebagai campuran tambahan	Kas		Rp 200.000
2025-01-18	T07	Upah tenaga kerja persemaian & pencabutan bibit	Beban Tenaga Kerja	Rp 400.000	
2025-01-18	T07	Upah tenaga kerja persemaian & pencabutan bibit	Kas		Rp 400.000
2025-01-25	T08	Upah buruh tanam (tandur) manual lahan 1,5 ha	Beban Tenaga Kerja	Rp 800.000	
2025-01-25	T08	Upah buruh tanam (tandur) manual lahan 1,5 ha	Kas		Rp 800.000
2025-02-05	T09	Pembelian pestisida tahap pertama (antisipasi hama)	Beban Pestisida & Obat	Rp 200.000	
2025-02-05	T09	Pembelian pestisida tahap pertama (antisipasi hama)	Kas		Rp 200.000
2025-02-15	T10	Upah tenaga kerja penyiangkan (membersihkan gulma)	Beban Tenaga Kerja	Rp 500.000	
2025-02-15	T10	Upah tenaga kerja penyiangkan (membersihkan gulma)	Kas		Rp 500.000
2025-02-25	T11	Pembelian obat-obatan tanaman susulan (hama sulit sembuh)	Beban Pestisida & Obat	Rp 200.000	
2025-02-25	T11	Pembelian obat-obatan tanaman susulan (hama sulit sembuh)	Kas		Rp 200.000
2025-03-10	T12	Pembelian pestisida/vitamin jelang masa bunting padi	Beban Pestisida & Obat	Rp 100.000	
2025-03-10	T12	Pembelian pestisida/vitamin jelang masa bunting padi	Kas		Rp 100.000

Gambar 30. Tampilan Jurnal Umum.

2.6.5 Buku Besar

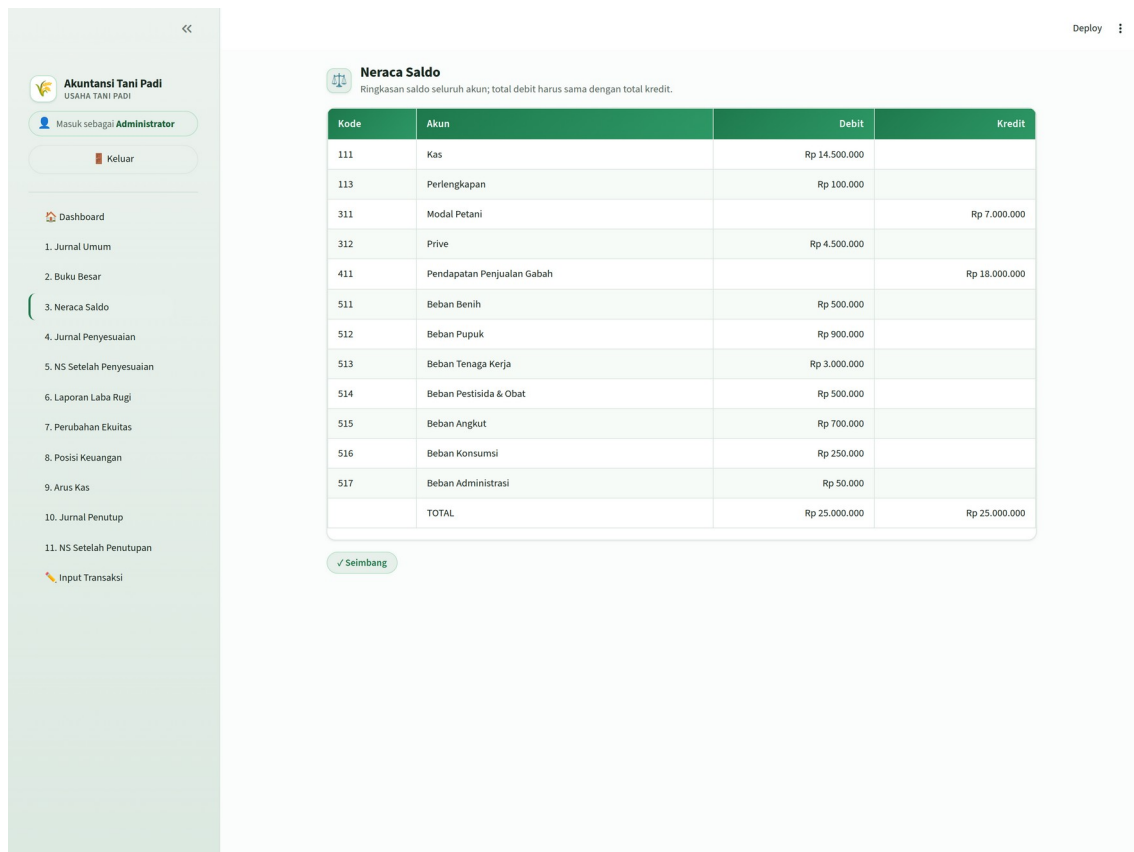
Buku Besar mengelompokkan transaksi berdasarkan masing-masing akun sehingga saldo setiap akun dapat dilihat dengan jelas.



Gambar 31. Tampilan Buku Besar.

2.6.6 Neraca Saldo

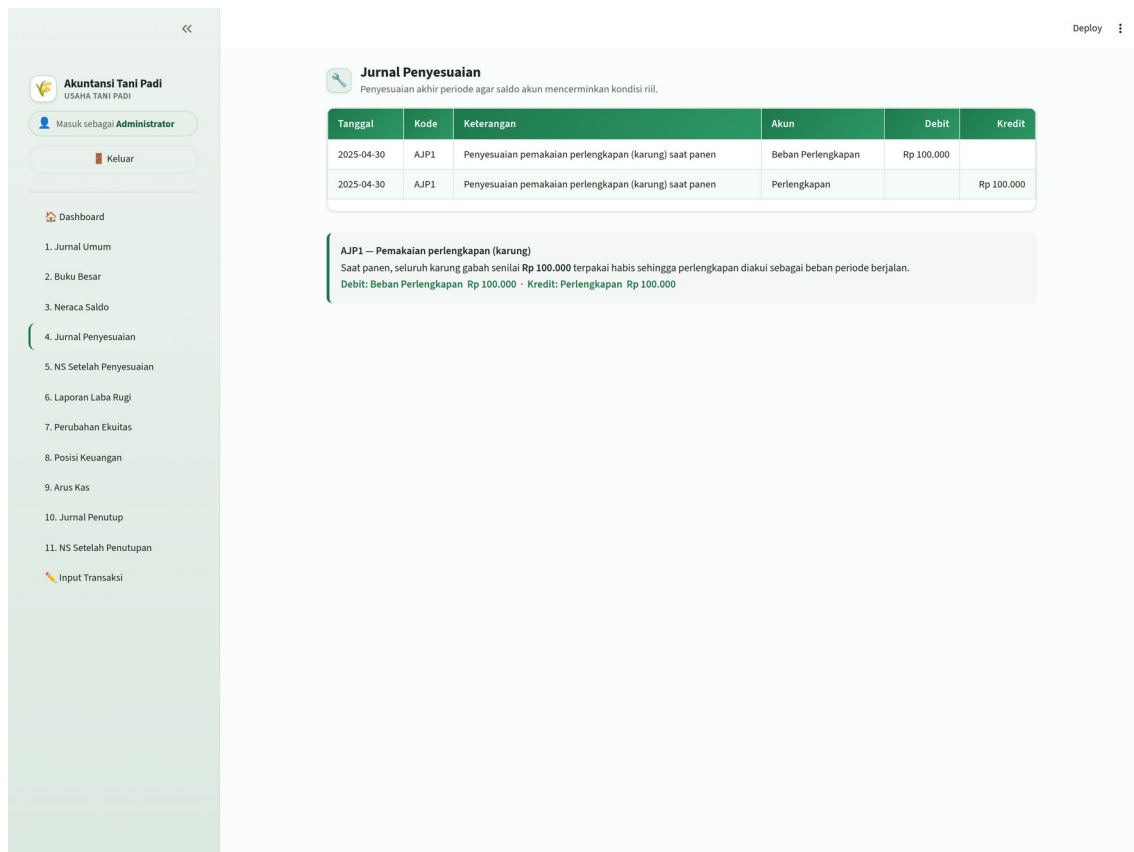
Neraca Saldo menampilkan saldo akhir seluruh akun dalam dua kolom (debit dan kredit) yang totalnya harus seimbang.



Gambar 32. Tampilan Neraca Saldo.

2.6.7 Jurnal Penyesuaian

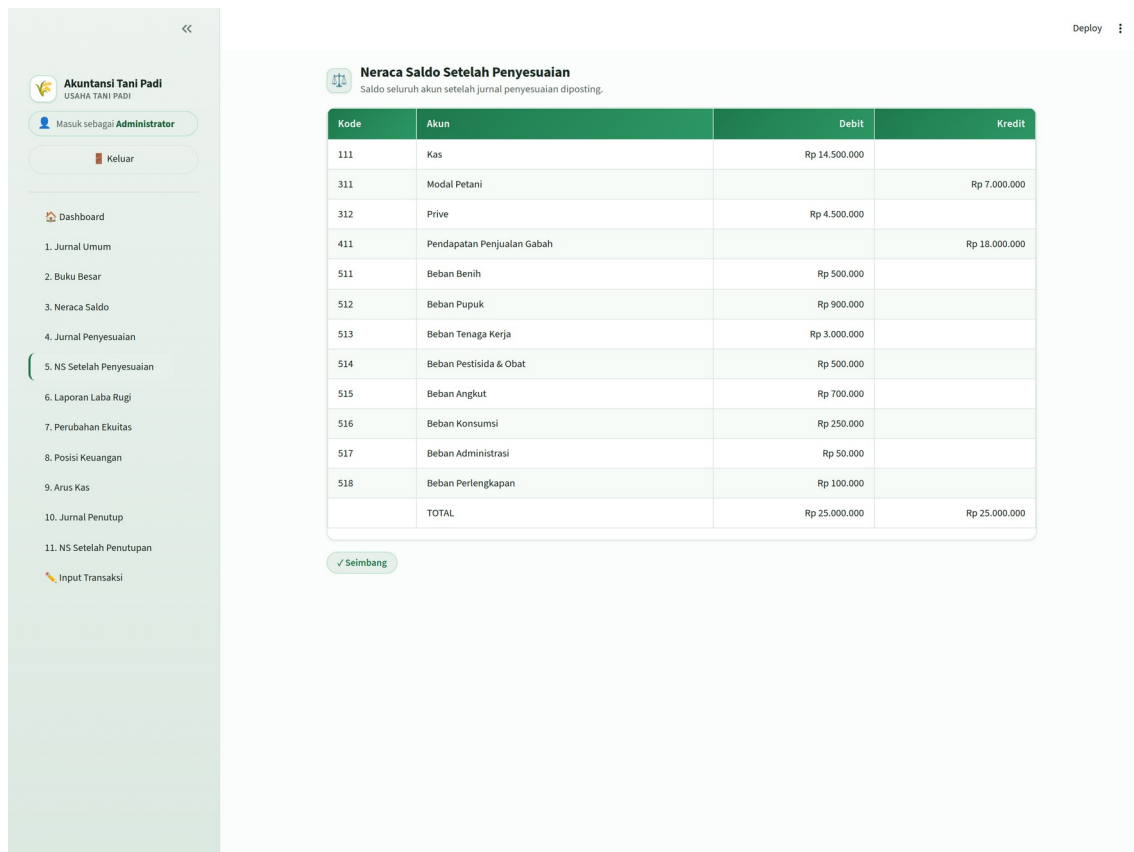
Jurnal Penyesuaian memuat penyesuaian akhir periode, misalnya penyusutan peralatan dan pemakaian perlengkapan.



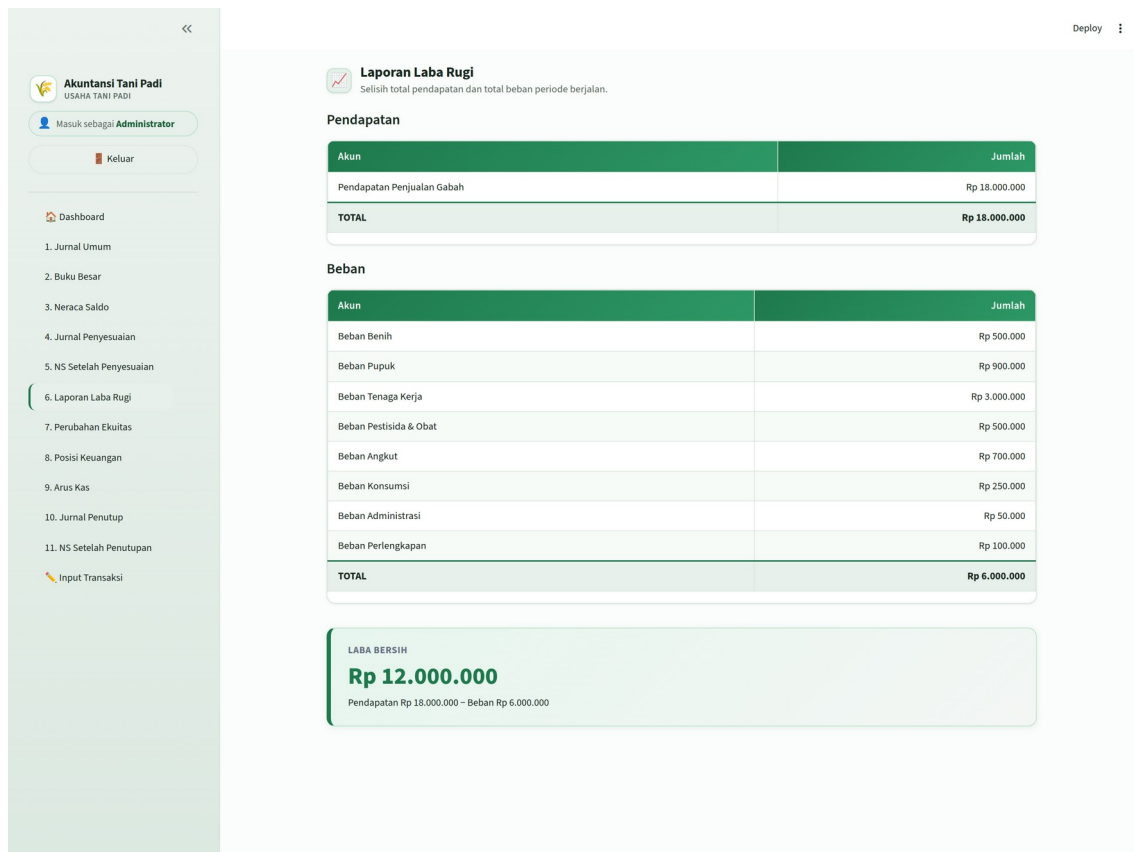
Gambar 33. Tampilan Jurnal Penyesuaian.

2.6.8 Neraca Saldo Setelah Penyesuaian

Laporan ini menampilkan saldo akun setelah jurnal penyesuaian diterapkan.



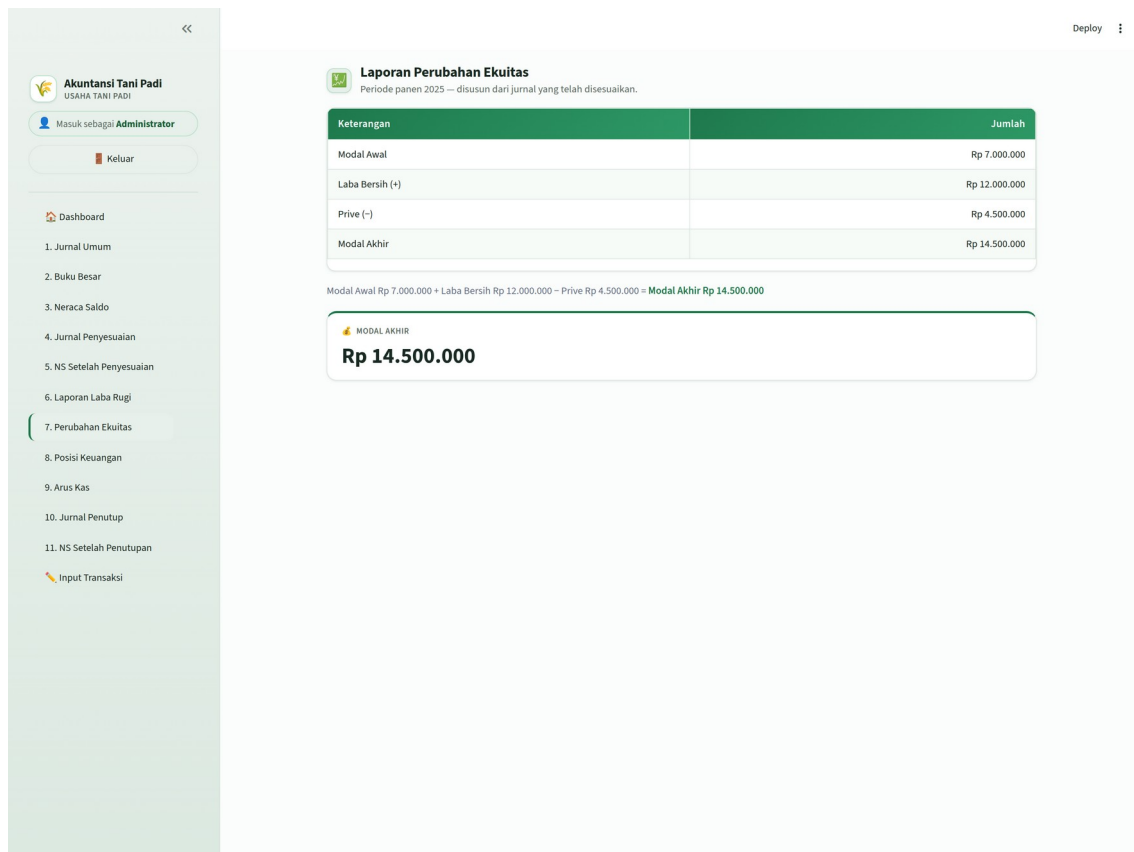
2.6.9 Laporan Laba Rugi



Gambar 35. Tampilan Laporan Laba Rugi.

2.6.10 Laporan Perubahan Ekuitas

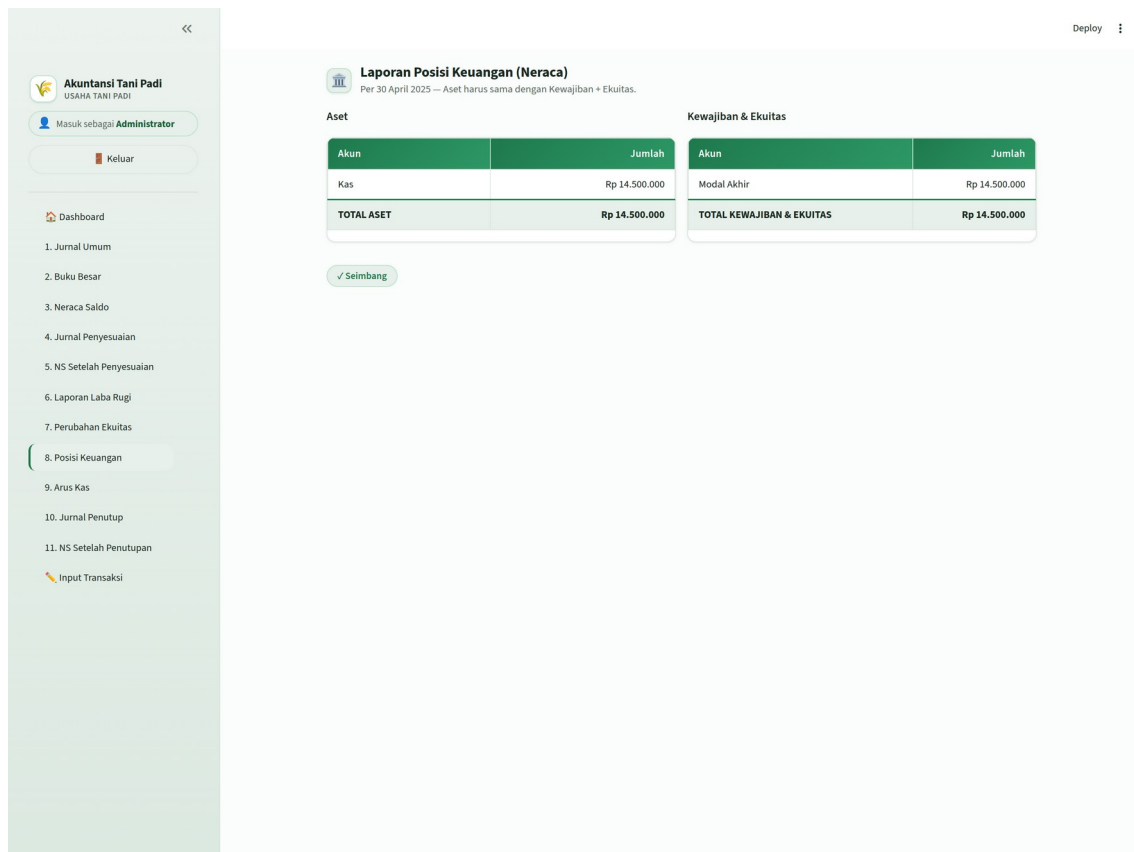
Laporan ini menunjukkan perubahan modal pemilik dari modal awal hingga modal akhir setelah memperhitungkan laba dan prive.



Gambar 36. Tampilan Laporan Perubahan Ekuitas.

2.6.11 Laporan Posisi Keuangan (Neraca)

Laporan Posisi Keuangan menampilkan aset, serta ekuitas pada akhir periode, di mana total aset sama dengan total ekuitas.



Gambar 37. Tampilan Laporan Posisi Keuangan.

2.6.12 Laporan Arus Kas

Laporan Arus Kas merangkum aliran kas masuk dan keluar sehingga saldo kas akhir dapat diketahui.

<<
Deploy

Akunansi Tani Padi

USANA TANI PADI

Masuk sebagai **Administrator**

Keluar

- Dashboard
-
- 1. Jurnal Umum
- 2. Buku Besar
- 3. Neraca Saldo
- 4. Jurnal Penyesuaian
- 5. NS Setelah Penyesuaian
- 6. Laporan Laba Rugi
- 7. Perubahan Ekuitas
- 8. Posisi Keuangan
- 9. Arus Kas
- 10. Jurnal Penutup**
- 11. NS Setelah Penutupan
-
- Input Transaksi

Jurnal Penutup

Memindahkan saldo akun nominal ke modal agar bersaldo nol.

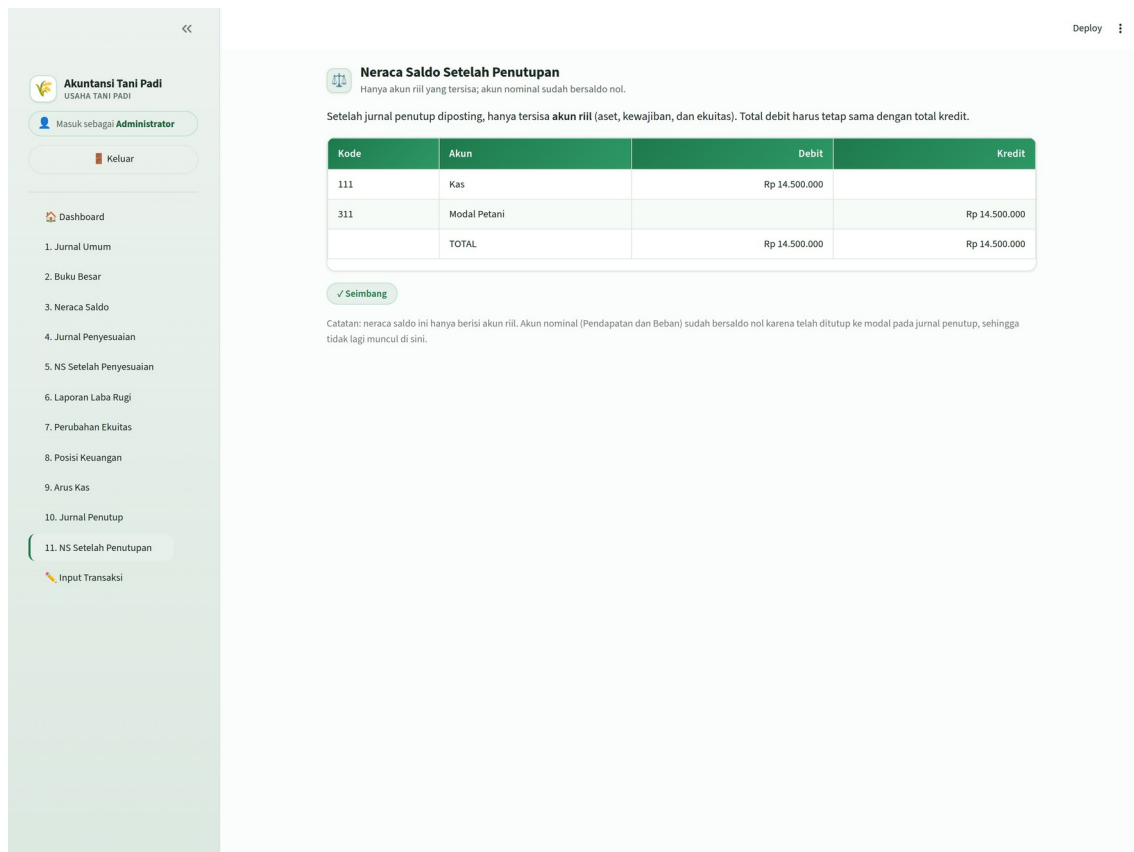
Jurnal penutup memindahkan saldo **akun nominal** (pendapatan, beban, dan prive) ke modal pada akhir periode, sehingga akun nominal kembali bersaldo nol untuk periode berikutnya.

Tanggal	Kode	Keterangan	Akun	Debit	Kredit
2025-04-30	JP1	Menutup akun pendapatan ke ikhtisar	Pendapatan Penjualan Gabah	Rp 18.000.000	
2025-04-30	JP1	Menutup akun pendapatan ke ikhtisar	Ikhtisar Laba Rugi		Rp 18.000.000
2025-04-30	JP2	Menutup akun beban ke ikhtisar	ikhtisar Laba Rugi	Rp 6.000.000	
2025-04-30	JP2	Menutup akun beban ke ikhtisar	Beban Benih		Rp 500.000
2025-04-30	JP2	Menutup akun beban ke ikhtisar	Beban Pupuk		Rp 900.000
2025-04-30	JP2	Menutup akun beban ke ikhtisar	Beban Tenaga Kerja		Rp 3.000.000
2025-04-30	JP2	Menutup akun beban ke ikhtisar	Beban Pestisida & Obat		Rp 500.000
2025-04-30	JP2	Menutup akun beban ke ikhtisar	Beban Angkut		Rp 700.000
2025-04-30	JP2	Menutup akun beban ke ikhtisar	Beban Konsumsi		Rp 250.000
2025-04-30	JP2	Menutup akun beban ke ikhtisar	Beban Administrasi		Rp 50.000
2025-04-30	JP2	Menutup akun beban ke ikhtisar	Beban Perlengkapan		Rp 100.000
2025-04-30	JP3	Menutup ikhtisar laba rugi ke modal (laba)	ikhtisar Laba Rugi	Rp 12.000.000	
2025-04-30	JP3	Menutup ikhtisar laba rugi ke modal (laba)	Modal Petani		Rp 12.000.000
2025-04-30	JP4	Menutup akun prive ke modal	Modal Petani	Rp 4.500.000	
2025-04-30	JP4	Menutup akun prive ke modal	Prive		Rp 4.500.000

Empat langkah penutupan:

- JP1 — Tutup pendapatan.** Seluruh akun pendapatan didebit, lawannya dikreditkan ke akun **Ikhtisar Laba Rugi**.
- JP2 — Tutup beban.** Seluruh akun beban dikredit, lawannya didebitkan ke akun **Ikhtisar Laba Rugi**.
- JP3 — Tutup ikhtisar ke modal.** Selisih ikhtisar (laba bersih Rp 11.500.000) dipindahkan ke akun **Modal Petani**.
- JP4 — Tutup prive ke modal.** Saldo prive Rp 4.500.000 mengurangi **Modal Petani**.

2.6.14 Neraca Saldo Setelah Penutupan



Gambar 40. Tampilan Neraca Saldo Setelah Penutupan.

2.6.15 Input Transaksi

Menu Input Transaksi menyediakan formulir untuk mencatat transaksi baru. Pengguna mengisi tanggal, keterangan, serta akun debit dan kredit. Aplikasi akan menolak penyimpanan bila total debit dan kredit tidak seimbang.

Akuntansi Tani Padi

USANA TANI PADI

Masuk sebagai Administrator

Keluar

Dashiboard

1. Jurnal Umum

2. Buku Besar

3. Neraca Saldo

4. Jurnal Penyesuaian

5. NS Setelah Penyesuaian

6. Laporan Laba Rugi

7. Perubahan Ekuitas

8. Posisi Keuangan

9. Arus Kas

10. Jurnal Penutup

11. NS Setelah Penutupan

Input Transaksi

Input Transaksi

Setiap transaksi terdiri dari satu baris debit dan satu baris kredit. Nominal debit harus sama dengan nominal kredit agar jurnal seimbang.

Tanggal
2026/05/31

Keterangan
Contoh: Pembelian benih padi

Baris Debit

Akun Debit
111 - Kas

Nominal Debit
0

Baris Kredit

Akun Kredit
111 - Kas

Nominal Kredit
0

Simpan Transaksi

Daftar Transaksi

Tanggal	Kode	Keterangan	Akun	Debit	Kredit
2025-01-05	T01	Setoran modal awal pemilik ke kas usaha tani	Kas	Rp 7.000.000	
2025-01-05	T01	Setoran modal awal pemilik ke kas usaha tani	Modal Petani		Rp 7.000.000
2025-01-07	T02	Pembelian benih padi Mekongga 10 kantong (50 kg)	Beban Benih	Rp 500.000	
2025-01-07	T02	Pembelian benih padi Mekongga 10 kantong (50 kg)	Kas		Rp 500.000
2025-01-10	T03	Upah tenaga kerja pembajakan & persiapan lahan 1,5 ha	Beban Tenaga Kerja	Rp 1.000.000	
2025-01-10	T03	Upah tenaga kerja pembajakan & persiapan lahan 1,5 ha	Kas		Rp 1.000.000
2025-01-12	T04	Pembelian pupuk Urea Pusri gelombang pertama	Beban Pupuk	Rp 400.000	
2025-01-12	T04	Pembelian pupuk Urea Pusri gelombang pertama	Kas		Rp 400.000
2025-01-13	T05	Pembelian pupuk TSP untuk dasar tanaman	Beban Pupuk	Rp 300.000	

2.6.16 Ubah Transaksi

Akuntransi Tani Padi

USANA TANI PADI

Masuk sebagai Administrator

Keluar

Dashboard

1. Jurnal Umum

2. Buku Besar

3. Neraca Saldo

4. Jurnal Penyesuaian

5. NS Setelah Penyesuaian

6. Laporan Laba Rugi

7. Perubahan Ekuitas

8. Posisi Keuangan

9. Arus Kas

10. Jurnal Penutup

11. NS Setelah Penutupan

Input Transaksi

Edit TransaksiT21

Akun dipilih dari bagan akun resmi. Debit harus sama dengan kredit.

Tanggal

2026/05/31

Keterangan

Penjualan gabah tunai tambahan (contoh panduan)

Baris Debit

Akun Debit

Nominal Debit

111 - Kas

500000

Baris Kredit

Akun Kredit

Nominal Kredit

411 - Pendapatan Penjualan Gabah

500000

Simpan PerubahanBatal

Daftar Transaksi

Tanggal	Kode	Keterangan	Akun	Debit	Kredit
2025-01-05	T01	Setoran modal awal pemilik ke kas usaha tani	Kas	Rp 7.000.000	
2025-01-05	T01	Setoran modal awal pemilik ke kas usaha tani	Modal Petani		Rp 7.000.000
2025-01-07	T02	Pembelian benih padi Mekongga 10 kantong (50 kg)	Beban Benih	Rp 500.000	
2025-01-07	T02	Pembelian benih padi Mekongga 10 kantong (50 kg)	Kas		Rp 500.000
2025-01-10	T03	Upah tenaga kerja pembajakan & persiapan lahan 1,5 ha	Beban Tenaga Kerja	Rp 1.000.000	
2025-01-10	T03	Upah tenaga kerja pembajakan & persiapan lahan 1,5 ha	Kas		Rp 1.000.000
2025-01-12	T04	Pembelian pupuk Urea Pusri gelombang pertama	Beban Pupuk	Rp 400.000	
2025-01-12	T04	Pembelian pupuk Urea Pusri gelombang pertama	Kas		Rp 400.000
2025-01-13	T05	Pembelian pupuk TSP untuk dasar tanaman	Beban Pupuk	Rp 300.000	
2025-01-13	T05	Pembelian pupuk TSP untuk dasar tanaman	Kas		Rp 300.000

BAB III PENUTUP

3.1 Kesimpulan

Aplikasi Akuntansi Tani Padi berhasil menerapkan sistem informasi akuntansi yang lengkap untuk kebutuhan pencatatan keuangan usaha tani padi. Aplikasi mampu mencatat transaksi dengan sistem berpasangan, memvalidasi keseimbangan debit dan kredit, serta menyajikan siklus akuntansi penuh dalam sebelas tahap laporan, mulai dari jurnal umum hingga neraca saldo setelah penutupan.

Dengan antarmuka berbasis web yang sederhana dan tema yang konsisten, aplikasi ini memudahkan pengguna tanpa latar belakang akuntansi yang mendalam untuk mengelola keuangan usahanya. Penggunaan tipe Decimal dan penyimpanan nilai uang sebagai teks menjamin akurasi perhitungan, sementara pengamanan kata sandi dengan PBKDF2-HMAC-SHA256 memberikan perlindungan pada data akun pengguna.

3.2 Saran

16. Menambahkan fitur ekspor laporan ke format PDF atau Excel agar laporan mudah dibagikan dan diarsipkan.
17. Menyediakan fitur multi-periode sehingga pengguna dapat membandingkan kinerja keuangan antarmusim tanam.
18. Menambahkan grafik visual pada dasbor untuk mempermudah pembacaan tren pendapatan dan beban.
19. Mempertimbangkan penyimpanan berbasis daring (cloud) agar data dapat diakses dari berbagai perangkat secara aman.